
LoRa Alliance

End Device Certification Requirements for EU863-870 MHz ISM Band Devices

**Version 1.6.2
August 2023**

Table of Contents

1. Introduction.....	9
1.1. Scope of LoRaWAN Certification	9
1.2. LoRaWAN Certification Process.....	9
1.3. Changes to functional test for LoRaWAN1.0.1 v1.2 from LoRaWAN1.0 v1.0	9
1.4. Changes to functional test for LoRaWAN1.0.2 v1.5 from LoRaWAN1.0.1 v1.2	10
1.5. Changes to functional test for LoRaWAN1.0.2 v1.6 from LoRaWAN1.0.2 v1.5	10
2. Functional Test Description for LoRaWAN Certification	12
2.1. Activation and Deactivation Pre and Post Test	13
2.1.1. Certification Application Activation	13
2.1.1.a. Test Procedure Message Sequence Chart	14
2.1.2. Certification Application Deactivation	14
2.1.2.a. Test Procedure Message Sequence Chart	14
2.2. Over the Air Activation	14
2.2.1. Pre-Join Behaviour.....	15
2.2.1.a. Test Procedure Message Sequence Chart	16
2.2.2. Join Accept with DLSettings.....	16
2.2.2.a. Test Procedure Message Sequence Chart	18
2.2.3. Join Accept with Delay Settings on RX2 window.....	19
2.2.3.a. Test Procedure Message Sequence Chart	20
2.2.4. Join Accept with CFList	20
2.2.4.a. Test Procedure Message Sequence Chart	22
2.2.5. DevNonce verification for Join Request	23
2.2.5.a. Test Procedure Message Sequence Chart	23
2.3. Activation by Personalization	25
2.4. Certification Application (Test Mode) Functionality	25
2.4.1. Default Setting Tests.....	25
2.4.1.a. Channel Plan and Usage	25
2.4.1.a.i. Test Procedure Message Sequence Chart	26
2.4.1.b. Cryptography:.....	26
2.4.1.b.i. AES Encryption	26
2.4.1.b.i.1. Test Procedure Message Sequence Chart	26
2.4.1.b.ii. Message Integrity Code	26

2.4.1.b.ii.1.	Test Procedure Message Sequence Chart	27
2.4.1.c.	Downlink Error Rate	27
2.4.1.c.i.	Test Procedure Message Sequence Chart	27
2.4.1.d.	Receive Window Timing	27
2.4.1.d.i.	Test Procedure Message Sequence Chart	28
2.4.1.e.	Frame Sequence Number	28
2.4.1.e.i.	Uplink Sequence Number	28
2.4.1.e.i.1.	Test Procedure Message Sequence Chart	29
2.4.1.e.ii.	Downlink Sequence Number	29
2.4.1.e.ii.1.	Test Procedure Message Sequence Chart	30
2.4.1.e.iii.	Downlink Sequence Number Rollover	30
2.4.1.e.iii.1.	Test Procedure Message Sequence Chart	31
2.4.2.	Confirmed Frames	31
2.4.2.a.	Confirmed Uplinks	31
2.4.2.a.i.	Test Procedure Message Sequence Chart	32
2.4.2.b.	Confirmed Uplink Retransmission	32
2.4.2.b.i.	Test Procedure Message Sequence Chart	33
2.4.2.c.	Confirmed Downlinks	35
2.4.2.c.i.	Test Procedure Message Sequence Chart	35
2.4.2.d.	Confirmed Downlink Retransmission	35
2.4.2.d.i.	Test Procedure Message Sequence Chart	36
2.5.	MAC Command Tests	36
2.5.1.	DevStatusReq	36
2.5.1.a.i.	Test Procedure Message Sequence Chart	37
2.5.2.	NewChannelReq	37
2.5.2.a.	Read-only default channels	37
2.5.2.b.	Addition of a channel	37
2.5.2.c.	Removal of a channel	37
2.5.2.d.	Addition and removal of multiple channels	37
2.5.2.e.	Invalid command processing - Frequency	37
2.5.2.f.	Invalid command processing – Data Range.....	38
2.5.2.g.	Removal of default channels – not allowed	38
2.5.2.h.	Test Procedure Message Sequence Chart	38

2.5.3.	DlChannelReq	43
2.5.3.a.	Test Procedure Message Sequence Chart	43
2.5.4.	RXParamSetupReq	45
2.5.4.a.	Test Procedure Message Sequence Chart	45
2.5.5.	RXTimingSetupReq	48
2.5.5.a.	Test Procedure Message Sequence Chart	48
2.5.6.	TxParamSetupReq MAC command	50
2.5.6.a.	Test Procedure Message Sequence Chart	50
2.5.7.	LinkCheckReq tests	50
2.5.7.a.	Test Procedure Message Sequence Chart	50
2.5.8.	LinkADRRReq	51
2.5.8.a.	ADR Bit	51
2.5.8.a.i.	Test Procedure Message Sequence Chart	51
2.5.8.b.	TxPower	51
2.5.8.b.i.	Test Procedure Message Sequence Chart	51
2.5.8.c.	Uplink Channel Management	53
2.5.8.c.i.	Required data rates	53
2.5.8.c.i.1.	Test Procedure Message Sequence Chart	53
2.5.8.c.ii.	Optional data rates	55
2.5.8.c.ii.1.	Test Procedure Message Sequence Chart	55
2.5.8.c.iii.	Unsupported data rates	57
2.5.8.c.iii.1.	Test Procedure Message Sequence Chart	58
2.5.8.c.iv.	ChannelMask functionality	58
2.5.8.c.v.	Disable all Channels	59
2.5.8.c.v.1.	Test Procedure Message Sequence Chart for Channel Mask and Disable all Channels	59
2.5.8.d.	Redundancy	61
2.5.8.d.i.	Test Procedure Message Sequence Chart	61
2.5.8.e.	Data Rate Decay	63
2.5.8.e.i.	Test Procedure Message Sequence Chart	65
2.5.8.f.	Command Block Channel Management	66
2.5.8.f.i.	Successful LinkADRRReq block	66
2.5.8.f.i.1.	Test Procedure Message Sequence Chart	67
2.5.8.f.ii.	Unsuccessful LinkADRRReq block	67

2.5.8.f.ii.1.	Test Procedure Message Sequence Chart	68
2.5.9.	RX Window test	69
2.5.9.a.	RX1 Receive Window Test	69
2.5.9.a.i.	Test Procedure Message Sequence Chart	70
2.5.9.b.	RX2 Receive Window Test	73
2.5.9.b.i.	Test Procedure Message Sequence Chart	73
2.5.9.c.	RX1 and RX2 simultaneous frames	74
2.5.9.c.i.	Test Procedure Message Sequence Chart	74
2.5.9.d.	RX Oversized Payload	74
2.5.9.d.i.	Test Procedure Message Sequence Chart	75
2.5.9.e.	Maximum Allowed Payload	76
2.5.9.e.i.	Uplink Payload	76
2.5.9.e.i.1.	Max Payload via Echo	76
2.5.9.e.i.1.1.	Test Procedure Message Sequence Chart	77
2.5.9.e.i.2.	Oversized Payload via Echo	77
2.5.9.e.i.2.1.	Test Procedure Message Sequence Chart	78
2.5.10.	MAC Command(s) in App-Payload (FRMPayload)	79
2.5.10.a.	App-Payload Only (FPort = 0)	79
2.5.10.a.i.	Test Procedure Message Sequence Chart	79
2.5.10.b.	Frame Options Only (FPort != 0)	79
2.5.10.b.i.	Test Procedure Message Sequence Chart	80
2.5.10.c.	App-Payload and Frame Options	80
2.5.10.c.i.	Test Procedure Message Sequence Chart	81
2.5.11.	Incorrect MAC Commands	81
2.5.11.a.	Test Procedure Message Sequence Chart	82
3.	Notice of Use and Disclosure	85

Figures

Figure 1: Test Harness Architecture	12
---	----

Revision History

Version	Author	Updates	Date
1.0	D. Hunt	Initial Version 1.0 for LoRaWAN Version 1.0	30 November 2015
1.1	D. Hunt, M. Ridder, J. Nikula	Extended Packet Error Rate tests	23 February 2016
1.2	D. Hunt, M. Ridder, J. Nikula	Updated for LoRaWAN Version 1.0.1	22 June 2016
1.3	D. Hunt, M. Ridder, S. Mäkelä	Updated for LoRaWAN Version 1.0.2 and LoRaWAN Regional Parameters Version 1.0.	21 December 2016
1.4	D. Hunt	Minor change for ADRackReq test to have a tolerance of +/-2 packet and modification to Continuous Wave Mode definition. Certification Committee approved for Release	21 April 2017
1.5	D. Hunt	Minor update to TX output power setting in LinkADRReq MAC command to align with LoRaWAN Regional Parameters v1.0.2rB	21 June 2017
1.6	Nisha Bhaskaran	Derived from the India Cert Spec v1.1. Added Missing EU tests as listed and agreed by the Operators and Certification Committee. Converted the sequence charts from images to tables and added them to the individual sections itself.	24 March 2020
1.6.1	Nisha Bhaskaran	Bug fix for Section 2.5.9.c.i - for Bugzilla #590 and #640	03 August 2023
1.6.2	Nisha Bhaskaran	Section 2.5.9.c.i, 2.5.10.a.i and 2.5.10.b.i are updated such that RxCnt increments for only applicative downlinks	10 August 2023

Glossary

LoRaWAN	Protocol specification developed and maintained by the LoRa Alliance.
Certification Logo	LoRa Alliance defined logo that can be displayed on the Certified product and any documentation and marketing information about the End-Device.
LoRa Test House	Organization and corresponding facility accredited by the LoRa Alliance to perform Certification testing.
End Device	Device submitted to a LoRa Test House for Certification.

Abbreviations & Acronyms

ABP	Activation by Personalization
ADR	Adaptive Data Rate
AS	Application Server
ATH	Authorized Test House
AWG	Arbitrary Waveform Generator
DR	Data Rate
DUT	Device Under Test
ERP	Equivalent Radiated Power compared to a dipole antenna (expressed in dBd)
EIRP	Equivalent Isotropic Radiated Power: $ERP = EIRP - 2.15dB$ (expressed in dBi)
ETSI	European Telecommunications Standards Institute
FSK	Frequency Shift Keying modulation technique.
GW	Gateway
LCTT	LoRaWAN Certification Test Tool
MAC	Media Access Control
NS	Network Server
OTAA	Over-the-Air Activation
TCL	Test Control Layer of the Test Harness
TRP	Total Radiated Power

References

- [1] LoRaWAN Specifications 1.0.2.
- [2] LoRaWAN Regional Parameters Specification 1.0.2 revB.
- [3] LoRaWAN Certification Protocol Specification 1.0.0.

1. Introduction

This document specifies the minimum testing requirements for an End-Device to be designated “**LoRaWAN Certified**”. LoRaWAN Certification will confirm that the End-Device meets the Functional Requirements of the *LoRaWAN Specification Version 1.0.2* [1] for the corresponding regional parameters as defined in the *LoRaWAN Regional Parameters Version 1.0.2 revB* [2].

1.1. Scope of LoRaWAN Certification

The scope of this test specification is limited to validating compliant implementation of the LoRaWAN protocol.

For LoRaWAN Certification it is not mandatory that the End-Device has all the Regulatory approvals, but these will be required before the product can be sold or operated in the respective countries. Intended or otherwise, the inevitable variability of performance and quality of the radio implementation among End-Devices is too high to allow normalized-, practical evaluation. RF performance measurement, whether radiated or conducted, is therefore considered out of scope for the tests described herein. The RF performance of the End-Device will be tested solely as part of the RF Performance Testing, which will be a separate test conducted at the Authorized Test House to evaluate the RF performance of the device as part of LoRaWAN Certification testing.

1.2. LoRaWAN Certification Process

A party seeking LoRaWAN Certification for their End-Device must be a member of the LoRa Alliance in good standing and only Test Houses designated accredited by the LoRa Alliance may perform the tests described herein to earn it.

The Authorized Test Houses must communicate the complete results to the LoRa Alliance. If the device has passed all mandatory tests, the LoRa Alliance will issue a certificate for the End-Device with respect to a version of this document and in turn corresponding versions of [1] and [2]. Additionally, the Alliance will publish both the status and a results summary on its web site along with data for any optional features tested.

1.3. Changes to functional test for LoRaWAN1.0.1 v1.2 from LoRaWAN1.0 v1.0

- ADRACKReq change: the device will further lower its data rate every ADR_ACK_DELAY, not only one step lower.
- Simultaneous MAC commands: If MAC commands are simultaneously present in the payload and options fields, the device shall ignore this frame
- LinkADRs: The DUT shall not give a channel mask ACK, if the server-provided channel mask requires all channels to be disabled.
- RXParamSetupAns: Is to be added in FOpt of all uplinks until a downlink is received (before: included only in the next uplink)
- RXTimingSetupAns: Is to be added in FOpt of all uplinks until a downlink is received (before: included only in the next uplink)
- Join Accept: change of MIC calculation (before: DLSettings field not included)
- Testing for Over The Air Activation added. DUT is activated using different Join-accept fields.

1.4. Changes to functional test for LoRaWAN1.0.2 v1.5 from LoRaWAN1.0.1 v1.2

- DIChannelReq: New MAC command was added
- Block of LinkADRReq commands: If a block of LinkADRReq is included in a frame, all of them should be processed before rejecting them. E.g. It should be possible to firstly disable all channels and then enable some.
- TX output power: The TX output power indicated in the command is to be considered the maximum transmit power the device may operate at. An end-device will acknowledge a successful command which can specify a higher transmit power than the DUT is capable of, but should then operate at its maximum possible power.
- Frequency hopping: The frequency hopping is to be tested by the test system, when for example a single channel is added. In V1.0.1 only one specific frequency was checked after adding; now all three mandatory channels will be waited for.
- ADRAckReq: Updated sequence charts. The ADRAckReq bit is checked for the 65th uplink frame after the last DL. The lower datarate is checked for the 97th and the 129th frame after the last DL.
- A new command for continuous wave (CW) transmission added to the End Device Test Application Specification.

1.5. Changes to functional test for LoRaWAN1.0.2 v1.6 from LoRaWAN1.0.2 v1.5

- Derived from the LoRaEndDeviceCertificationIndiaVer1.1.
- Included all the missing tests identified by the European Operators and enhancements to the existing tests.
- The terminology for the Test Server has been changed to Test Control Layer (TCL) in this document.
- Message sequence charts were converted from images to text format.

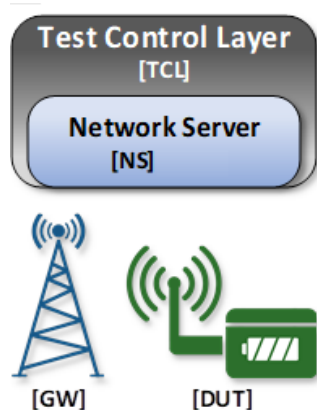
Sl. No.	Test description	Section number in this document
1	Device deactivation	2.1.2
2	Tests for Activation by Personalization	2.3
3	OTAA Pre-Join and duration between successive Join requests. Record data rate	2.2.1
4	Join Accept with empty CFList	2.2.4
5	Check 16 channels conformity	2.5.2.d
6	DevStatusAns - Check coherence of battery and Margin values	2.5.1
7	MAC command rejection in payload / options field	2.5.9.c

8	Unconfirmed DL test	2.2.2, 2.4.1.b.i
9	LinkCheckReq and behaviour after LinkCheck answer	2.5.7
10	Check that RXTimingSetupAns is repeated until first DL	2.5.5
11	LinkADDRReq test TXPower	2.5.8.b
12	ADR Redundancy. No duplicated messages after MAC command and NBTrans > 1	2.5.8.d
13	ADR Ack Req / Data Rate Decay	2.5.8.e
14	Incorrect MAC payload	2.5.11
15	AppNonce and DevNonce behavior in v1.0.x - random value or monotonically increasing counter?	2.2.5
16	TxParamSetupReq	2.5.6
17	NewChannelReq with wrong parameters	2.5.2.e and 2.5.2.f
18	Confirmed UL retransmission stop	2.4.2.b
19	RxParamSetupReq with wrong parameters	2.5.4

Table 1: EU- Missing tests added to this document

2. Functional Test Description for LoRaWAN Certification

The list of tests specified below reflects the functional requirements of a Class-A End-Device as defined by the LoRaWAN Specification Version 1.0.2. The tests are conducted in a test harness generally comprised of:



- A Test Control Layer [TCL]
- A LoRaWAN Network Server [NS]
- A 16 channel LoRaWAN gateway [GW]
- The End-Device Under Test [DUT]

Figure 1: Test Harness Architecture

Implementation of this harness architecture is expected to vary among test houses. The Test Control Layer [TCL] is assumed to be a framework of automated scripts and tools that manipulates the Network Server [NS] to facilitate the tests. Specifically, the **TCL** drives events in the harness, controlling application and network-control content of downlinks. It also decrypts, inspects and validates content of uplinks sent by the **DUT**. This allows test coverage to include:

- Cryptography
- Timing of the **DUT** Receive Windows
- Frequency Channel usage and Data Rate adaptation
- Max Payload handling

For brevity reasons, this document makes procedural reference to only the **TCL**, **NS**, the **DUT**.

The LoRaWAN gateway [GW] and **DUT** are collocated in an RF-isolated environment, provisioned as necessary for reliable bi-directional communication. It is nonetheless expected that both the **DUT** and **GW** will not receive every frame intended for reception. The **TCL** should make reasonable effort to accommodate this inevitability.

The **DUT** is required to implement a certification application (see Section 2.1.1 for details) as an alternative to its intended-, normal application. The RF-isolated environment mentioned above should mitigate any potential interference.

Testing occurs to certify the **DUT** for each supported activation method, be it over-the-air activation (OTAA), activation-by-personalized (ABP), or both.

Between each test section below, the **TCL** will return the **DUT** to its default test state: all channels enabled, uplinking as quickly as possible, ideally every 5 seconds (or faster), but worse case every 10 seconds, an unconfirmed frame whose payload reflects the device's current Downlink counter value.

The **TCL** must verify the following throughout the course of this certification test suite:

- The **DUT**'s uplinks' size must respect the maximum allowed uplink size for the data rate used.
- The length of the **DUT**'s uplinks based on the expected content to ensure no extraneous and unnecessary content is present.

2.1. Activation and Deactivation Pre and Post Test

Test Note1: Section 2.1.1 – Certification Application Activation tests - must be executed as the first test when executing a single or multiple tests. After the complete set of tests are executed, Section 2.1.2 – Certification Application Deactivation - must be the last step to be executed. During the execution of the complete test suite, the Test Mode does not need to be activated or deactivated.

Test Note2: The **TCL** sends frames only on the RX2 window of the DUT for all tests in this document, unless specified otherwise in the Sequence Table Test cases.

The DUT must support either over-the-air (OTA) activation or activation by personalization (ABP) or both. If the device supports both OTAA and ABP, the device vendor must provide 2 separate devices, one supporting OTAA and the other supporting ABP method of activation to the Authorised Test House (ATH). The ATH will use the appropriate device for each activation test.

2.1.1. Certification Application Activation

After initial power-up – and activation if the **DUT** supports OTAA instead of ABP – the **DUT** must transmit an uplink packet as soon as possible (recommended within 10 seconds). Contents of this "I'm alive" packet are unimportant. The **TCL** replies to this packet with a downlink payload message of 0x01010101 sent to port 224, enabling the cert-application.

Upon activation of the test mode, the **DUT** must send an unconfirmed uplink every 5 seconds with its payload containing the current downlink counter. This value is expected to be 0x0000 when the cert-application is activated.

Verify that

- Test Mode is activated on the device
- The **DUT** must disable any duty cycle restrictions as well as suspend its normal application software.

For more details on the Certification Application implementation, refer to the Certification Protocol Specification [3].

2.1.1.a. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
	If an OTA device, Step 1 and 2 are required. Step 1 and 2 must be skipped for ABP devices.			
1	DUT sends a Join-Request frame	→	DataRate (DR) – Any allowed DR, refer [2]	
2	TCL sends a Join-Accept frame	←		
3	DUT sends Unconfirmed or Confirmed frame FCntUp = X (where X is any random number)	→		
4	The TCL sends Unconfirmed frame	←	FPort = 224 Payload = 0x01010101	Test Mode ON
5	DUT sends Unconfirmed frame without any duty cycle restrictions FCntUp = X + 1	→	FPort = 224 Payload = 0x0000	Cert-App Active

2.1.2. Certification Application Deactivation

In this test, the **TCL** will send a downlink payload message of 0x00 over port 224 which should disable the test mode and the **DUT** must switch back to its default operation.

Verify that

- Test mode is disabled and the device switches back to its default operation

2.1.2.a. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame FCntUP = X	→		
2	The TCL sends Unconfirmed frame	←	FPort = 224 Payload = 0x00	Test Mode OFF
3	If DUT sends Unconfirmed/Confirmed frame- FCntUP = X + 1	→	FPort = any allowed port except 224	Cert-App Disabled

2.2.Over the Air Activation

This test must be performed if the **DUT** supports over the air activation. The test verifies the correct functionality of the over-the-air activation.

Furthermore, the fields within the JoinAccept message (DLSettings and RxDelay) used to modify the data rates and receive window timing are tested to verify compliance.

2.2.1. Pre-Join Behaviour

The **TCL** commands a reset, and the **DUT** must respond with a JoinRequest message. The **TCL** ignores the JoinRequest messages until the **DUT** responds on all the 3 default channels (Channel 0, 1 and 2). After this, the **TCL** responds with a JoinAccept message. The **TCL** waits for a maximum of 9 JoinRequest messages before responding with a JoinAccept message.

Verify

- **DUT** sends JoinRequest messages until the **TCL** responds with a JoinAccept message.
- All the 3 default channels – Channel 0, 1 and 2 must be used at least once in these requests.
- Record the Data Rate of the initial Join Requests.
- The duration between the Join Requests is greater than the JOIN_ACCEPT_DELAY2 which is 6 seconds for EU region.
- Check if the Major version in the MAC header is correct and the RFU bits are set to 0
- **DUT** successfully joins the network.

Once the device begins uplinking, the **TCL** activates the certification application by sending a downlink to the DUT on RX1, allowing transition to the next test.

2.2.1.a. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame FCntUP = n	→		
2	The TCL sends Unconfirmed frame	←	CMD Reset FPort = 224 Payload = 0x06	Device Reinitializes [Not Joined]
3	DUT sends a maximum of 9 JoinRequest messages (until JoinRequest messages are sent on all 3 default channels)	→	- All JoinRequest messages must be sent on Channel 0, 1 and 2 at least once. - Duration between previous JoinReq and next JoinReq > 6 seconds for all Join Requests - Record the Data Rate of the Join requests - Major version is correct - RFU bits = 0.	
4	TCL ignores all the JoinRequest messages until all 3 default channels are used for the JoinRequest messages	←		
5	DUT sends JoinRequest message again	→		
6	TCL sends Join Accept response	←		Join accepted
7	DUT sends Unconfirmed frame or Confirmed frame FCntUP = i (where i = 0 or 1)	→	DR – Any	
8	The TCL sends Unconfirmed frame	←	CMD Test ON FPort 224 Payload 0x01010101	Test Mode ON
9	DUT sends Unconfirmed frame FCntUP = i + 1	→	DR – Any FPort = 224 Payload = 0x0000	
10	Wait for a maximum of 5 * 3 (number of channels configured) = 15 uplink packets to be sent, i.e. until all channels are used at least once.	→	FPort = 224	All channels must be used at least once

2.2.2. Join Accept with DLSettings

After the **TCL** triggers a JoinRequest, the **DUT** starts the Join procedure for over-the-air activation. The **TCL** responds with a JoinAccept message with RX1DROffset = 2 and RX2DataRate = SF9BW125. After the join procedure succeeds, the **TCL** downlinks an echo command targeting

the RX1 window to which the **DUT** must respond correctly. Next the **TCL** downlinks an echo command targeting the RX2 window to which the **DUT** must respond correctly.

Verify

- **DUT** successfully joins the network
- **DUT** implements $RX1DROffset = 2$ after processing the Join Accept
- **DUT** implements $RX2DataRate = SF9BW125$ after processing the Join Accept

2.2.2.a. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame FCntUP = n	→		
2	The TCL sends Unconfirmed frame	←	CMD Reset FPort = 224 Payload = 0x06	Device Reinitializes [Not Joined]
3	DUT sends JoinReq	→	DR – Any	
4	TCL sends Join Accept response	←	RX1 DR Offset = 2 RX2 DR = 3	Join accepted with modified parameters
5	DUT sends Unconfirmed frame or Confirmed frame FCntUP = i (where i = 0 or 1)	→	DR – Any	
6	The TCL sends Unconfirmed frame	←	CMD Test ON FPort 224 Payload 0x01010101	Test Mode ON
7	DUT sends Unconfirmed frame FCntUP = i + 1	→	DR – Any FPort = 224 Payload = 0x0000 No ACK is sent for the previous Unconfirmed frame	
8	The TCL sends Unconfirmed frame on RX1 window	←	DR – Any + 2 CMD Echo Req FPort 224 Payload 0x04010203	
9	DUT sends Unconfirmed frame	→	DR – Any CMD Echo Reply FPort 224 Payload 0x04020304	RX1 reply sent
10	The TCL sends Unconfirmed frame on RX2 window	←	RX2 DR - 3 CMD Echo Req FPort 224 Payload 0x040A0B0C	
11	DUT sends Unconfirmed frame	→	DR – Any CMD Echo Reply FPort 224 Payload 0x040B0C0D	RX2 reply sent
12	Wait for a maximum of 5 * 3 (number of channels configured) = 15 uplink packets to be sent, i.e. until all channels are used at least once.	→	FPort = 224	All channels must be used at least once

2.2.3. Join Accept with Delay Settings on RX2 window

After the **TCL** triggers a JoinRequest, the **DUT** starts the Join procedure for over-the-air activation as above. The **TCL** responds with a JoinAccept message containing Delay Settings over the RX2 window, such that RX1 and subsequently RX2 timing is increased (at least 2 seconds are recommended). The **TCL** sends an echo command targeted to RX1 to which the **DUT** must respond correctly. The **TCL** repeats this same downlink test against the RX2 window, to which the **DUT** must respond correctly.

Verify

- **DUT** successfully joins the network
- **DUT** implements the new (non-default) Delay Settings
- **DUT** restores the default settings for RX1DRoffset and RX2DataRate

2.2.3.a. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device – TCL	Message	
1	DUT sends Unconfirmed frame FCntUP = n	→		
2	The TCL sends Unconfirmed frame on RX2 window	←	CMD Reset FPort = 224 Payload = 0x06	Device Reinitializes [Not Joined]
3	DUT sends JoinReq	→	DR – Any	
4	TCL sends Join Accept response on RX2 window	←	RX Delay = n 2 ≤ n ≤ 15	Join accepted with modified parameters
5	DUT sends Unconfirmed frame or Confirmed frame FCntUP = i (where i = 0 or 1)	→	DR – Any	
6	The TCL sends Unconfirmed frame	←	CMD Test ON FPort 224 Payload 0x01010101	Test Mode ON
7	DUT sends Unconfirmed frame FCntUP = i + 1	→	FPort = 224 Payload = 0x0000	
8	The TCL sends Unconfirmed frame on RX1 window	←	RX1Delay = n seconds CMD Echo Req FPort 224 Payload 0x04010203	
9	DUT sends Unconfirmed frame	→	CMD Echo Reply FPort 224 Payload 0x04020304	RX1 delay reply sent
10	The TCL sends Unconfirmed frame on RX2 window	←	RX2Delay = n + 1 seconds RX2DataRate = default, refer [2] CMD Echo Req FPort 224 Payload 0x040A0B0C	
11	DUT sends Unconfirmed frame	→	DR – Any CMD Echo Reply FPort 224 Payload 0x040B0C0D	RX2 delay reply sent

2.2.4. Join Accept with CFList

After the **TCL** triggers a JoinRequest, the **DUT** starts the Join procedure for over-the-air activation as above. The **TCL** responds with a JoinAccept message containing an additional single channel in the CFList field. The **DUT** must use this additional channel together with the default channels within the following uplinks. The **TCL** again triggers the **DUT** to send a JoinRequest. The **TCL** responds with a JoinAccept message

containing an empty CFList. The **DUT** must not use the additional channel added earlier.

Verify

- **DUT** successfully joins the network
- **DUT** uses the new channel in its random selection of frequencies
- **DUT** removes the additional channel added earlier.

2.2.4.a. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame FCntUP = n	→		
2	The TCL sends Unconfirmed frame	←	CMD Reset FPort = 224 Payload = 0x06	Device Reinitializes [Not Joined]
3	DUT sends JoinReq	→	DR – Any	
4	TCL sends Join Accept response	←	CFList = add single channel	Join accepted
5	DUT sends Unconfirmed frame or Confirmed frame FCntUP = i (where i = 0 or 1)	→	DR – Any	
6	The TCL sends Unconfirmed frame	←	CMD Test ON FPort 224 Payload 0x01010101	Test Mode ON
7	DUT sends Unconfirmed frame FCntUP = i + 1	→	FPort = 224 Payload = 0x0000	
8	Wait until the new channel which was added has been used at least once. Wait for a maximum of 5* (number of channels configured) uplink packets to be sent.	→	FPort = 224	DUT adds the additional channel to its default channel plan
9	The TCL sends Unconfirmed frame	←	CMD Reset FPort = 224 Payload = 0x06	Device Reinitializes [Not Joined]
10	DUT sends JoinReq	→	DR – Any	
11	TCL sends Join Accept response	←	CFList = empty list	Join accepted
12	DUT sends Unconfirmed frame or Confirmed frame FCntUP = i (where i = 0 or 1)	→	DR – Any	
13	The TCL sends Unconfirmed frame	←	CMD Test ON FPort 224 Payload 0x01010101	Test Mode ON
14	DUT sends Unconfirmed frame FCntUP = i + 1	→	FPort = 224 Payload = 0x0000	DUT removes the additional channel from its default channel plan
15	Wait for 5* (number of channels configured) uplink packets to be sent.	→	- Default channels 0, 1 and 2 are used at least once. - The channel removed is not used. - FPort = 224	DUT removes the additional channel from its default channel plan

2.2.5. DevNonce verification for Join Request

This test is applicable and must be performed only if the DUT supports the "Lorawan-1.0.x-join-synch-issues-remedies-v1.0.0". If not supported, the test can be skipped.

After the **TCL** triggers a JoinRequest, the **DUT** starts the Join procedure by over-the-air activation as above. The **TCL** must respond with a JoinAccept message and the **DUT** must join the network. The **TCL** must again trigger a JoinRequest in a similar manner two more times and respond with JoinAccept messages both times.

Verify

- The DevNonce value sent by the DUT in the JoinRequest must be incremented in each JoinRequest.
- **DUT** accepts the JoinAccept messages and joins the network each time

2.2.5.a. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame FCntUP = n	→		
2	The TCL sends Unconfirmed frame	←	CMD Reset FPort = 224 Payload = 0x06	Device Reinitializes [Not Joined]
3	DUT sends JoinReq	→	DevNonce = x DR – Any	DevNonce is a random value
4	TCL sends Join Accept response	←		Join accepted
5	DUT sends Unconfirmed frame or Confirmed frame FCntUP = i (where i = 0 or 1)	→	DR – Any	
6	The TCL sends Unconfirmed frame	←	CMD Test ON FPort 224 Payload 0x01010101	Test Mode ON
7	DUT sends Unconfirmed frame FCntUP = i + 1	→	DR – Any FPort = 224 Payload = 0x0000	
8	The TCL sends Unconfirmed frame	←	CMD Reset FPort = 224 Payload = 0x06	Device Reinitializes [Not Joined]
9	DUT sends JoinReq	→	DevNonce = x + y DR – Any	DevNonce must be incremented
10	TCL sends Join Accept response	←		Join accepted
11	DUT sends Unconfirmed frame or Confirmed frame FCntUP = i (where i = 0 or 1)	→	DR – Any	
12	The TCL sends Unconfirmed frame	←	CMD Test ON FPort 224 Payload 0x01010101	Test Mode ON
13	DUT sends Unconfirmed frame FCntUP = i + 1	→	DR – Any FPort = 224 Payload = 0x0000	
14	The TCL sends Unconfirmed frame	←	CMD Reset FPort = 224 Payload = 0x06	Device Reinitializes [Not Joined]
15	DUT sends JoinReq	→	DevNonce = x + y + z DR – Any	DevNonce must be incremented
16	TCL sends Join Accept response	←		Join accepted
17	DUT sends Unconfirmed frame or Confirmed FCntUP = i (where i = 0 or 1)	→	DR – Any	
18	The TCL sends Unconfirmed frame	←	CMD Test ON FPort 224 Payload 0x01010101	Test Mode ON

19	DUT sends Unconfirmed frame FCntUP = i + 1	→	DR – Any FPort = 224 Payload = 0x0000	
----	---	---	---	--

2.3. Activation by Personalization

This test must be performed if the **DUT** supports activation by personalization.

For this test, the Test Lab must use the device provided by the device vendor to test the ABP activation functionality.

After initial power-up, the certification application of the **DUT** must be enabled as specified in Section 2.1.1.

Verify

- **DUT** successfully joins the network.

The Test Procedure Message Sequence Chart for this test is the same as Section 2.1.1.

2.4. Certification Application (Test Mode) Functionality

2.4.1. Default Setting Tests

The **TCL** will test the basic functionality of the **DUT** certification application using the periodically transmitted uplink packets containing the downlinks-received counter value (represented in sequence charts as 'RxCnt') and using the 0x04 echo command. The purpose of these tests is to detect implementation errors in the certification application early, instead of failing later tests.

2.4.1.a. Channel Plan and Usage

Check that all channels configured are used by the **DUT** within a maximum of 5*(number of channels configured) uplink transmissions.

2.4.1.a.i. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame FCntUP = n	→	FPort = 224	
2	Wait for a maximum of 5 * (number of channels configured) uplink packets to be sent, i.e. until all channels are used at least once.	→	FPort = 224	All channels must be used at least once

2.4.1.b. Cryptography:

Verifies that AES encryption and message integrity code algorithms are correctly implemented by the **DUT**.

2.4.1.b.i. AES Encryption

TCL will send multiple 0x04 echo commands with varying length payloads to the **DUT**. The echo-replies from the **DUT** are verified to contain the same payload where each byte is incremented by one. This test will only use echo commands whose payload is less than or equal to the maximum allowed payload of the **DUT**'s current uplink DR. Frame counter must increase for every received package. Oversized payloads are tested separately in Section **Error! Reference source not found.**

2.4.1.b.i.1. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame FCntUP = n	→	Payload (RxCnt) = x FPort = 224	
2	The TCL sends Unconfirmed frames	←	CMD Echo Req (repeat i times- where i = 1 to 5) FPort = 224 Payload = 0x04 (Various)	
3	DUT sends Unconfirmed frames FCntUp = n + i	→	CMD Echo Reply (repeat i times) FPort = 224 Payload = 0x04 (Various) No ACK is sent for the previous Unconfirmed frame	Multiple Echo replies sent and no ACK is sent for Unconfirmed frames
4	DUT sends Unconfirmed frames	→	Payload (RxCnt) = x + 5 FPort = 224	

2.4.1.b.ii. Message Integrity Code

TCL will send packets with purposely invalid message integrity codes. The **DUT** must ignore these packets. Frame counter must still increase +1 per each received package.

2.4.1.b.ii.1. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame FCntUP = n	→	Payload (RxCnt) = x FPort = 224	
2	The TCL sends Unconfirmed frames	←	CMD Echo Req (repeat i times - where i = 1 to 5) FPort = 224 Payload = 0x04 (Various) MIC invalid	
3	DUT sends Unconfirmed frames FCntUp = n + i	→	Payload (RxCnt) = x FPort = 224	Invalid MIC packets ignored

2.4.1.c. Downlink Error Rate

This test establishes a baseline indication of the **DUT**'s ability to sufficiently receive downlinks in near-ideal RF conditions. **Sufficient Reception** is defined as 95% success of (valid) downlinks sent being received and processed by the **DUT**. Given the provisioned RF test environment described in Section 1.1, a maximum reception failure rate of 5% is considered acceptable for certification.

The **TCL** will test the RX2 window, using its default data rate SF12BW125 (DR0), sending 60 unconfirmed downlinks of the maximum length allowed by the downlink DR, per [2]. At least 57 of the 60 downlinks must be received - equivalent to a packet error rate of 5% or lower. The echo command is not used as part of the **TCL**'s downlinks in this test. Instead, the unconfirmed command (0x03) is supplemented with random bytes such that the maximum payload length is achieved. The default payload of the uplinks - **DUT**'s downlink received counter value (**not** the same as the downlink's SeqNo) - will indicate whether each preceding downlink was received.

2.4.1.c.i. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame	→	Payload (RxCnt) = x FPort = 224	
2	The TCL sends Unconfirmed frames on RX2 window (60 times)	←	DR = 0 Payload = 0x03 (MaxLen for RX2 DR) FPort = 224	
3	DUT sends Unconfirmed frames	→	Payload (RxCnt) ≤ x + 60 Payload (RxCnt) ≥ x + 57 FPort = 224	Reception failure rate ≤ 5%

2.4.1.d. Receive Window Timing

The **DUT** must demonstrate capability to receive downlinks within ±20 µs of either RECEIVE_DELAY1 or RECEIVE_DELAY2. The **TCL** sends downlink packets at both -20 and +20 µs

offsets from the nominal receive window location and verifies that the packets are properly received by **DUT**.

2.4.1.d.i. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame FCntUp = n	→	FPort = 224 Payload (RxCnt) = x	
2	The TCL sends Unconfirmed frame on RX1 window	←	CMD Echo Req FPort = 224 Payload = 0x04XX TxDelay = 1000020 μs	
3	DUT sends Unconfirmed frame FCntUp = n + 1	→	CMD Echo Reply Payload = 0x04YY FPort = 224	Received RX1 packet (+20 μs Delay)
4	DUT sends Unconfirmed frame FCntUp = n + 2	→	Payload (RxCnt) = x + 1 FPort = 224	
5	The TCL sends Unconfirmed frame on RX2 window	←	CMD Echo Req FPort = 224 Payload = 0x04XX TxDelay = 2000020 μs	
6	DUT sends Unconfirmed frame FCntUp = n + 3	→	CMD Echo Reply Payload = 0x04YY FPort = 224	Received RX2 packet (+20 μs Delay)
7	The TCL sends Unconfirmed frame on RX1 window	←	CMD Echo Req FPort = 224 Payload = 0x04XX TxDelay = 999980 μs	
8	DUT sends Unconfirmed frame FCntUp = n + 4	→	CMD Echo Reply Payload = 0x04YY FPort = 224	Received RX1 packet (-20 μs Delay)
9	DUT sends Unconfirmed frame FCntUp = n + 5	→	Payload (RxCnt) = x + 1 FPort = 224	
10	The TCL sends Unconfirmed frame on RX2 window	←	CMD Echo Req FPort = 224 Payload = 0x04XX TxDelay = 1999980 μs	
11	DUT sends Unconfirmed frame FCntUp = n + 6	→	CMD Echo Reply Payload = 0x04YY FPort = 224	Received RX2 packet (-20 μs Delay)

2.4.1.e. Frame Sequence Number

This test verifies the **DUT** properly handles the frame sequence numbers that are used to prevent replay attacks on the communication.

2.4.1.e.i. Uplink Sequence Number

TCL verifies that consecutive uplink packets have incrementing sequence numbers.

2.4.1.e.i.1. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame FCntUp = x	→	Payload (RxCnt) = y FPort = 224	
2	After 5 seconds, DUT sends Unconfirmed frame FCntUp = x + 1	→	Payload (RxCnt) = y FPort = 224	

2.4.1.e.ii. Downlink Sequence Number

TCL sends several packets with decreasing sequence number i such that:

$$1 \leq i < \text{Current FCntDown}$$

The **DUT** must ignore downlinks whose sequence numbers are less than its current-, internal downlink counter value.

Next the **TCL** sends several packets with a sequence number that increases by a value i where:

$$1 < i < \text{MAX_FCNT_GAP}$$

or

$$1 < i < 16384$$

The **DUT** must accept packets whose sequence numbers are greater than its current-, internal downlink counter value by less than or equal to the *MAX_FCNT_GAP*, updating its internal downlink counter to the received sequence number.

2.4.1.e.ii.1. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame FCntUp = n	→	Payload (RxCnt) = x FPort = 224	
2	The TCL sends Unconfirmed frame FCntDown = a	←	FPort = 224 Payload = 0x03	
3	DUT sends Unconfirmed frame FCntUp = n + 1	→	Payload (RxCnt) = x + 1 FPort = 224	
4	The TCL sends Unconfirmed frame FCntDown = a – i (where i = [1, a-1]) <i>Repeat y times</i>	←	FPort = 224 Payload = 0x03	
5	DUT sends Unconfirmed frame FCntUp = n + 2 + y <i>Repeat y times</i>	→	Payload (RxCnt) = x + 1 FPort = 224	DUT ignores y downlinks with FCnt < current downlink counter
6	DUT sends Unconfirmed frame FCntUp = z	→	Payload (RxCnt) = x + 1 = A FPort = 224	
7	The TCL sends Unconfirmed frame FCntDown = a + i (where i = [1, 16384]) <i>Repeat y times</i>	←	FPort = 224 Payload = 0x03	
8	DUT sends Unconfirmed frame FCntUp = z + y <i>Repeat y times</i>	→	Payload (RxCnt) = A + y FPort = 224	DUT accepts downlinks within the MAX_FCNT_GAP

2.4.1.e.iii. Downlink Sequence Number Rollover

The **TCL** sends several packets with sequence numbers that intentionally increase the **DUT**'s downlink frame counter to a value α where:

$$65535 - \left(\frac{MAX_FCNT_GAP}{2} \right) \leq \alpha < 65535$$

or

$$57343 \leq \alpha < 65535$$

The **TCL** then sends a downlink whose sequence number β where:

$$65535 < \beta \leq 73727$$

In the case the **DUT** employs a 16-bit counter, the **TCL** then sends a downlink whose sequence number β where:

$$0 \leq \beta \leq 8192$$

The **DUT** must accept downlinks with sequence numbers beyond a 16-bit roll-over boundary that remain within the *MAX_FCNT_GAP*.

2.4.1.e.iii.1. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame FCntUp = n	→	Payload (RxCnt) = x FPort = 224	
2	The TCL sends Unconfirmed frame FCntDown = w + i (where i = [1, 16384]) <i>Repeat y times</i>	←	FPort = 224 Payload = 0x03	TCL sends y downlinks to force FCntDown to A [57343 <= A <= 65535]
3	DUT sends Unconfirmed frame <i>Repeat y times</i> FCntUp = n + y	→	Payload (RxCnt) = x + y FPort = 224	DUT accepts all downlinks and increments FCntUp to A
4	DUT sends Unconfirmed frame FCntUp = n + y + 1	→	Payload (RxCnt) = x + y FPort = 224	
6	The TCL sends Unconfirmed frame FCntDown = B (where 65535 <= B <= 73727. If DUT is using a 16-bit counter, 0 <= B <= 8192)	←	FPort = 224 Payload = 0x03	
7	DUT sends Unconfirmed frame <i>Repeat y times</i> FCntUp = n + y + 2	→	Payload (RxCnt) = x + y + 1 FPort = 224	DUT accepts downlink beyond the rollover

2.4.2. Confirmed Frames

This test verifies that the **DUT** properly handles confirmed frames as both the sender (uplinks) and receiver (downlinks).

2.4.2.a. Confirmed Uplinks

The **TCL** sends the 0x02 command to the **DUT**. Verify all subsequent uplinks from the **DUT** are message type *ConfirmedUp*.

2.4.2.a.i. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame FCntUp = n	→	Payload = y DR – X FPort = 224	
2	The TCL sends Unconfirmed frame	←	CMD Test FPort 224 Payload 0x02	
3	DUT sends Confirmed frame FCntUp = n + 1	→	Payload = y + 1 DR – X FPort = 224	Confirmed frame sent
4	The TCL sends Unconfirmed frame	←	Acknowledge No FPort and no payload	
5	DUT sends Confirmed frame FCntUp = n + 2	→	Payload = y + 2 DR – X FPort = 224	Confirmed frame sent
6	The TCL sends Unconfirmed frame	←	Acknowledge CMD Test FPort 224 Payload 0x02	
7	DUT sends Confirmed frame FCntUp = n + 3	→	Payload = y + 3 DR – X FPort = 224	Confirmed frame sent
8	The TCL sends Unconfirmed frame	←	Acknowledge CMD Test FPort 224 Payload 0x03	
9	DUT sends Unconfirmed frame FCntUp = n + 4	→	Payload = y + 4 FPort = 224	Switch back to Unconfirmed frame

2.4.2.b. Confirmed Uplink Retransmission

Note: This test must not be performed if the **DUT** does not support uplink re-transmissions for Confirmed frames. The maximum number of re-transmission of uplinks supported by the **DUT** must be obtained from the Certification Questionnaire.

The **TCL** intentionally does not acknowledge a confirmed uplink from the **DUT**. The **DUT** must retransmit the same packet with the same sequence number.

Verify

- The sequence number of retransmitted uplinks remains the same as the initially unacknowledged uplink.

- The data rate of the retransmitted uplink packet is less than or equal to the previous unacknowledged uplink. i.e. if the data rate of the previous uplink was DR2, then the retransmitted uplink could be DR0 thru DR2 but not DR3 or DR4.

Also verify that the **DUT** ceases retransmission when:

- It receives acknowledgement in a downlink.
- It has exhausted the maximum number of retransmits per its implementation.

2.4.2.b.i. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame FCntUp = n	→	Payload = y DR – X FPort = 224	
2	The TCL sends Unconfirmed frame	←	CMD Test FPort 224 Payload 0x02	
3	DUT sends Confirmed frame FCntUp = n + 1	→	Payload = y + 1 DR – X FPort = 224	Confirmed frame sent
4	The TCL stops sending downlinks to DUT			
5	DUT sends Confirmed frame FCntUp = n + 1	→	Payload = y + 1 DR ≤ X FPort = 224	
6	The TCL sends Unconfirmed frame	←	Acknowledge No FPort and no payload	
7	DUT sends Confirmed frame FCntUp = n + 2	→	Payload = y + 2 DR ≤ X FPort = 224	Confirmed frame sent on receiving ACK
8	The TCL sends Unconfirmed frame	←	Acknowledge CMD Test FPort 224 Payload 0x02	
9	DUT sends Confirmed frame FCntUp = n + 3	→	Payload = y + 3 DR – X FPort = 224	Confirmed frame sent
10	The TCL stops sending downlinks to DUT and waits until maximum number of retries of uplink transmission is reached			
11	DUT sends Confirmed frame FCntUp = n + 3 Repeat for maximum number of retransmissions.	→	Payload = y + 3 DR ≤ X FPort = 224	After maximum number of retransmissions is reached, uplink transmission stops
12	DUT sends Confirmed frame FCntUp = n + 4	→	FPort = 224 Payload = y + 3	
13	The TCL sends Unconfirmed frame	←	Acknowledge CMD Test FPort 224 Payload 0x03	
14	DUT sends Unconfirmed frame FCntUp = n + 5	→	Payload = y + 4 FPort = 224	Switch back to Unconfirmed frame

2.4.2.c. Confirmed Downlinks

TCL sends a *ConfirmedDown* packet. Verify the **DUT** sets the *ACK* bit in the subsequent uplink.

2.4.2.c.i. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame		FPort = 224 Payload = y	
2	The TCL send Confirmed frame	←	CMD Test FPort 224 Payload 0x02	TCL sends Confirmed frame
3	DUT may send an empty Unconfirmed frame (Optional step)	→	ACK Bit = True FPort = Empty port	
4	DUT sends Confirmed frame FCntUp = n + 1	→	ACK Bit = True (only if step 3 is not sent) Payload = y + 1 DR = X FPort = 224	Confirmed frame sent with ACK Bit set
5	The TCL sends Unconfirmed frame	←	Acknowledge No FPort and no payload	
6	DUT sends Confirmed frame FCntUp = n + 2	→	Payload = y + 2 DR = X	Confirmed frame sent
7	The TCL sends Unconfirmed frame	←	Acknowledge CMD Test FPort 224 Payload 0x03	
8	DUT sends Unconfirmed frame FCntUp = n + 3	→	Payload = y + 3 FPort = 224	Switch back to Unconfirmed frame

2.4.2.d. Confirmed Downlink Retransmission

Repeat the above test, but the **TCL** ignores the acknowledgement in the subsequent uplink from the **DUT** and retransmits the confirmed downlink with the same sequence number. Verify the **DUT** acknowledges the retransmission of the confirmed downlink.

2.4.2.d.i. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame		FPort = 224 Payload = y	
2	The TCL send Confirmed frame FCntDown = x	←	CMD Test FPort 224 Payload 0x03	TCL sends Confirmed frame
3	DUT may send an empty Unconfirmed frame (Optional step)	→	ACK Bit = True FPort = Empty port	
4	DUT sends Unconfirmed frame FCntUp = n + 1	→	ACK Bit = True (only if step 3 is not sent) Payload = y + 1 FPort = 224	Confirmed frame sent with ACK Bit set
5	The TCL ignores ACK and re-transmits same Confirmed downlink FCntDown = x	←	CMD Test FPort 224 Payload 0x03	
6	DUT may send an empty Unconfirmed frame (Optional step)	→	ACK Bit = True FPort = Empty port	
7	DUT sends Unconfirmed frame FCntUp = n + 2	→	ACK Bit = True (only if step 6 is not sent) Payload = y + 2 FPort = 224	DUT acknowledges the retransmission

2.5. MAC Command Tests

The following tests will validate the **DUT**'s implementation of MAC command processing and the associated functional areas being controlled by the **TCL**. As previously stated, the **TCL** should allow for some reasonable amount of packet loss while facilitating tests. Specifically, where MAC commands are concerned, it is acceptable to retry commanding the device in the absence of a MAC command answer. Retries should be limited to a maximum of 5 attempts.

Test Note1: In this Section, for all MAC Command Tests, the Uplink payload can be sent in the FRMPayload field with Port = 0 or in the FOpts field in the header with Port = 224.

Test Note2: For each of the MAC Command, the MAC Command Payload Command Identifier (CID) must be obtained from [1]

2.5.1. DevStatusReq

TCL sends a *DevStatusReq* command to the **DUT**.

Verify that

- The **DUT** must reply with a 2-byte *DevStatusAns* packet.

- The signal-to-noise margin in the reply is a signed integer of 6 bits with a minimum value of -32 and maximum value of 31.

2.5.1.a.i. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame	→	FPort = 224	
2	The TCL sends Unconfirmed frame	←	CMD DeviceStatusReq Payload 0x06X FPort = 0	
3	DUT sends Unconfirmed frame	→	CMD DeviceStatusAns SNR Margin ≥ -32 and ≤ 31 Payload = 0x06X	DeviceStatusAns sent and encoded value tested

2.5.2. NewChannelReq

TCL sends a *NewChannelReq* command to the **DUT** for configuring new channel frequencies.

2.5.2.a. Read-only default channels

Channels 0 to 2 are mandatory default channels. The **TCL** will send *NewChannelReq* that tries to modify these channels. The **DUT** must reply with an unsuccessful *NewChannelAns* (0x0700). The **TCL** checks that the three default channels are used (as described in 2.4.1.a above).

2.5.2.b. Addition of a channel

TCL sends a MAC command to add a single new channel. The **DUT** must reply with a successful *NewChannelAns* and begin using the new channel in its random selection of frequencies. The frequency used for this test is 867.7 MHz, but if the frequency is outside the documented capabilities of the **DUT**, a different frequency can be used instead.

2.5.2.c. Removal of a channel

TCL sends *NewChannelReq* to set the previously configured channel to 0 MHz frequency. The **DUT** must reply with a successful *NewChannelAns* and stop using the additional channel in its uplink transmissions.

2.5.2.d. Addition and removal of multiple channels

TCL sends multiple MAC commands in a single packet to configure the additional channels. Multiple such frames must be sent to configure all the 13 additional channels and to remove them. As above, the **DUT** must reply with a successful *NewChannelAns* to each request. Verify that the DUT sends an uplink on each channel configured.

2.5.2.e. Invalid command processing - Frequency

TCL sends *NewChannelReq* command including an invalid frequency located out of the band (e.g. 910.3 MHz) to check that the **DUT** replies correctly.

2.5.2.f. Invalid command processing – Data Range

TCL sends *NewChannelReq* command including an invalid data range to check that the **DUT** replies correctly.

2.5.2.g. Removal of default channels – not allowed

TCL tries to remove the default channels. **DUT** must reject the command.

2.5.2.h. Test Procedure Message Sequence Chart

Step	Procedure	End Device - TCL	Message Sequence	Test Purpose
			Message	
1	DUT sends Unconfirmed frame	→	FPort = 224	
2	The TCL sends Unconfirmed frame	←	CMD NewChannelReq ChannelIndex = 2 Freq = X Payload = 0x07X	TCL attempts to modify default channels
3	DUT sends Unconfirmed frame	→	CMD NewChannelAns Payload = 0x0700	DUT shall not change its channel plan or transmission behaviour
4	The TCL sends Unconfirmed frame	←	CMD NewChannelReq ChannelIndex = 3 Freq = 867.7 MHz Payload = 0x07X	
5	DUT sends Unconfirmed frame	→	CMD NewChannelAns Payload = 0x0703	
6	Wait for a maximum of 5 * 4 (number of channels configured) = 20 uplink packets to be sent , i.e. until the channel configured is used at least once	→	The new channel configured must be used at least once. FPort = 224	New channel added
7	DUT sends Unconfirmed frame	→	FPort = 224	
8	The TCL sends Unconfirmed frame	←	CMD NewChannelReq Channel Index = 3 Freq = 0 MHz Payload = 0x07X	
9	DUT sends Unconfirmed frame	→	CMD NewChannelAns Payload = 0x0703	
10	Wait for 5 * 3 (number of channels configured) = 15 uplink packets to be sent	→	The channel removed must not be used FPort = 224	DUT returns to its default channel plan
11	DUT sends Unconfirmed frame	→	FPort = 224	
12	The TCL sends Unconfirmed frame	←	CMD NewChannelReq Channel Index = 3 Freq = 867.1 MHz Channel Index = 4 Freq = 867.3 MHz Channel Index = 5 Freq = 867.5 MHz Channel Index = 6 Freq = 867.7 MHz Channel Index = 7 Freq = 867.9 MHz Payload = 0x07X	

Step	Procedure	End Device - TCL	Message Sequence	Test Purpose
			Message	
13	DUT sends Unconfirmed frame	→	CMD NewChannelAns Payload = 0x07030x07030x07030x07030x0703 DUT uses 8 channels (3 default + 5 additional) for sending uplink messages	5 additional channels added
14	DUT sends Unconfirmed frame	→	FPort = 224	
15	The TCL sends Unconfirmed frame	←	CMD NewChannelReq Channel Index = 8 Freq = 867.9 MHz Channel Index = 9 Freq = 867.7 MHz Channel Index = 10 Freq = 867.5 MHz Channel Index = 11 Freq = 867.3 MHz Channel Index = 12 Freq = 867.1 MHz Payload = 0x07X	
16	DUT sends Unconfirmed frame	→	CMD NewChannelAns Payload = 0x07030x07030x07030x07030x0703 DUT uses 13 channels for sending uplink messages	5 more additional channels added
17	DUT sends Unconfirmed frame	→	FPort = 224	
18	The TCL sends Unconfirmed frame	←	CMD NewChannelReq Channel Index = 13 Freq = 868.5 MHz Channel Index = 14 Freq = 868.3 MHz Channel Index = 15 Freq = 868.1 MHz Payload = 0x07X	
19	DUT sends Unconfirmed frame	→	CMD NewChannelAns Payload = 0x07030x07030x0703 DUT uses 16 channels for sending uplink messages	3 more additional channels added
20	The TCL sends Unconfirmed frame	←	CMD LinkADDRReq ChMsk = 0xF000 Payload = 0x03X	
21	DUT sends Unconfirmed frame	→	CMD LinkADRAns Payload = 0x0307	Enable channel IDs 12-15
22	Wait for 5 * 4 (number of channels enabled) = 20 uplink packets to be sent	→	Only the 4 enabled channels must be used at least once, and the disabled channels must not be used FPort = 224	Checking channel usage

Step	Procedure	End Device - TCL	Message Sequence Message	Test Purpose
23	The TCL sends Unconfirmed frame	←	CMD LinkADDRReq ChMsk = 0x0F00 Payload = 0x03X	
24	DUT sends Unconfirmed frame	→	CMD LinkADRAns Payload = 0x0307	Enable channel IDs 8-11
25	Wait for 5 * 4 (number of channels enabled) = 20 uplink packets to be sent	→	Only the 4 enabled channels must be used at least once, and the disabled channels must not be used FPort = 224	Checking channel usage
26	The TCL sends Unconfirmed frame	←	CMD LinkADDRReq ChMsk = 0x00F0 Payload = 0x03X	
27	DUT sends Unconfirmed frame	→	CMD LinkADRAns Payload = 0x0307	Enable channel IDs 4-7
28	Wait for 5 * 4 (number of channels enabled) = 20 uplink packets to be sent	→	Only the 4 enabled channels must be used at least once, and the disabled channels must not be used FPort = 224	Checking channel usage
29	The TCL sends Unconfirmed frame	←	CMD LinkADDRReq ChMsk = 0x000F Payload = 0x03X	
30	DUT sends Unconfirmed frame	→	CMD LinkADRAns Payload = 0x0307	Enable channel IDs 0-3
31	Wait for 5 * 4 (number of channels enabled) = 20 uplink packets to be sent	→	Only the 4 enabled channels must be used at least once, and the disabled channels must not be used FPort = 224	Checking channel usage
32	The TCL sends Unconfirmed frame	←	CMD NewChannelReq Channel Index = 3 Freq = 0 MHz Channel Index = 4 Freq = 0 MHz Channel Index = 5 Freq = 0 MHz Channel Index = 6 Freq = 0 MHz Payload = 0x07X	
33	DUT sends Unconfirmed frame	→	CMD NewChannelAns Payload = 0x07030x07030x07030x0703	4 additional Channels removed
34	The TCL sends Unconfirmed frame	←	CMD NewChannelReq Channel Index = 7 Freq = 0 MHz Channel Index = 8 Freq = 0 MHz Channel Index = 9 Freq = 0 MHz Channel Index = 10 Freq = 0 MHz Payload = 0x07X	

Step	Procedure	End Device - TCL	Message Sequence Message	Test Purpose
35	DUT sends Unconfirmed frame	→	CMD NewChannelAns Payload = 0x07030x07030x07030x0703	4 additional Channels removed
36	The TCL sends Unconfirmed frame	←	CMD NewChannelReq Channel Index = 11 Freq = 0 MHz Channel Index = 12 Freq = 0 MHz Channel Index = 13 Freq = 0 MHz Channel Index = 14 Freq = 0 MHz Channel Index = 15 Freq = 0 MHz Payload = 0x07X	
37	DUT sends Unconfirmed frame	→	CMD NewChannelAns Payload = 0x07030x07030x07030x07030x 0703	Remaining 5 additional Channels removed
38	The TCL sends Unconfirmed frame	←	CMD NewChannelReq Channel Index = 3 Freq = 910.3 MHz Payload = 0x07X	
39	DUT sends Unconfirmed frame	→	CMD NewChannelAns Payload is NOT = 0x0703	DUT shall not add the channel due to invalid frequency
40	The TCL sends Unconfirmed frame	←	CMD NewChannelReq Channel Index = 3 Freq = 867.7 MHz DRRange = 0x88 Payload = 0x07X	
41	DUT sends Unconfirmed frame	→	CMD NewChannelAns Payload is NOT = 0x0703	DUT shall not add the channel due to invalid Data Range
42	The TCL sends Unconfirmed frame	←	CMD NewChannelReq Channel Index = 0 Freq = 0 MHz Channel Index = 1 Freq = 0 MHz Channel Index = 2 Freq = 0 MHz Payload = 0x07X	
43	DUT sends Unconfirmed frame	→	CMD NewChannelAns Payload is NOT 0x0703	Default channel not affected

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
44	Wait for $5 * 3$ (number of channels configured) = 15 uplink packets to be sent	→	Only the 3 default channels must be used at least once and the non-default channels must not be used FPort = 224	Checking channel usage

2.5.3. DIChannelReq

The **TCL** sends a MAC command to change the downlink frequency in Rx1 for an existing channel. The **TCL** tests RX1 downlink window using the new frequency and then restores the default values. The **TCL** waits for an uplink, while the MAC answer should be 0x0A03. Then the **TCL** sends a downlink with an invalid frequency and waits for the next uplink, while the MAC answer should not contain 0x0A03. Finally, standard settings are applied and tested again.

2.5.3.a. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame	→	FPort = 224	
2	The TCL sends Unconfirmed frame on RX1 window	←	CMD DChannelReq ChannelIndex = [0-2] Freq = X Payload = 0x0AX	
3	DUT sends Unconfirmed frame <i>Repeat 3 times until a downlink is received confirming the receipt of the DChannelAns</i>	→	CMD DChannelAns Payload = 0x0A03	
4	The TCL must wait until the DUT sends an uplink on the same channel configured in Step 2 and then sends Unconfirmed frame on RX1 window	←	CMD EchoReq FPort = 224 Payload = 0x04010203 Downlink Freq = X	
5	DUT sends Unconfirmed frame	→	CMD EchoReply FPort = 224 Payload = 0x04020304	Tests new downlink frequency settings
6	DUT sends Unconfirmed frame	→	FPort = 224	
7	The TCL sends Unconfirmed frame	←	CMD DChannelReq Channel Index = [0-2] Downlink Freq = default frequency Payload = 0x0AX	
8	DUT sends Unconfirmed frame <i>Repeat 3 times until a downlink is received confirming the receipt of the DChannelAns</i>	→	CMD DChannelAns Payload = 0x0A03	DUT returns to its default settings
9	The TCL must wait until the DUT sends an uplink on the same channel configured in Step 7 and then sends Unconfirmed frame	←	CMD EchoReq FPort = 224 Payload = 0x040A0B0C Downlink Freq = default frequency	
10	DUT sends Unconfirmed frame	→	CMD EchoReply FPort = 224 Payload = 0x040B0C0D	Tests default downlink frequency settings
11	DUT sends Unconfirmed frame	→	FPort = 224	
12	The TCL sends Unconfirmed frame	←	CMD DChannelReq Channel Index = [0-2] Downlink Freq = X (where X = invalid frequency) Payload = 0x0AX	
13	DUT sends Unconfirmed frame	→	CMD DChannelAns Payload is NOT = 0x0A03	Unsuccessful
14	DUT sends Unconfirmed frame	→	FPort = 224	
15	The TCL sends Unconfirmed frame	←	CMD DChannelReq Channel Index = 4 Downlink Freq = default frequency Payload = 0x0AX	

16	DUT sends Unconfirmed frame	→	CMD NewChannelAns Payload is NOT = 0x0A03	Unsuccessfu l
----	-----------------------------	---	--	------------------

2.5.4. RXParamSetupReq

The **TCL** sends a MAC command to configure new *RX1DRoffset*, *RX2DataRate* and *RX2Frequency*. The **DUT** must include a successful *RXParamSetupAns* in all subsequent uplinks until a downlink is received. The **TCL** will wait for more than one uplink containing the affirmative *RXParamSetupAns* of 0x0507. The **TCL** then sends a downlink and verifies the next uplink does not contain the *RXParamSetupAns* 0x0507. Both the RX1 and RX2 downlink windows are then tested using the new parameters. Finally, default settings are restored by way of MAC command and both RX windows tested again.

The **TCL** sends a MAC command with invalid *RX2DRoffset*, *RX2DataRate* and *RX2Frequency* that is not supported. Verify that the **DUT** ignores the command and the previous parameters are kept.

Note: It is at the discretion of the Test Tool vendor as to how many uplinks are sent before the **TCL** downlinks.

2.5.4.a. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame FCntUp = y	→	FPort = 224	
2	The TCL sends Unconfirmed frame	←	CMD LinkADRReq TxPwrIndx = 0 DataRate = 3 ChnMskCntl = 0 ChnMsk = 0x0007 NbTrans = 1 Payload = 0x03X	
3	DUT sends Unconfirmed frame	→	CMD LinkADRAns DataRate = 3 Payload = 0x0307	DataRate set to 3
4	The TCL sends Unconfirmed frame	←	CMD RxParamSetupReq RX1DROffset = [1-7] RX2DataRate = [0-5] RX2Frequency = X Payload = 0x05X	
5	DUT sends Unconfirmed frame <i>Repeat 3 times until a downlink is received confirming the receipt of the RxParamSetupAns</i> FCntUp >= y + 3	→	CMD RxParamSetupAns Payload = 0x0507	
6	The TCL sends Unconfirmed frame on RX1 window	←	CMD EchoReq FPort = 224 Payload = 0x04010203	
7	DUT sends Unconfirmed frame	→	CMD EchoReply FPort = 224 Payload = 0x04020304	Tested for new parameters on RX1
8	The TCL sends Unconfirmed frame on RX2 window	←	CMD EchoReq FPort = 224 Payload = 0x04121314 Downlink Frequency = X DataRate = [0-5]	
9	DUT sends Unconfirmed frame	→	CMD EchoReply FPort = 224 Payload = 0x04131415	Tested for new parameters on RX2
10	The TCL sends Unconfirmed frame	←	CMD RxParamSetupReq RX1DROffset = 0 RX2DataRate = 0 RX2Frequency = 869.525 Payload = 0x05X	
11	DUT sends Unconfirmed frame	→	CMD RxParamSetupAns Payload = 0x0507	Restored to default settings
12	The TCL sends Unconfirmed frame on RX1 window	←	CMD EchoReq FPort = 224 RX1DROffset = 0 Payload = 0x04010203	

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
13	DUT sends Unconfirmed frame	→	CMD EchoReply FPort = 224 Payload = 0x04020304	Default Rx1 Params verified
14	The TCL sends Unconfirmed frame on RX2 window	←	CMD EchoReq RX2DataRate = 0 RX2Frequency = 869.525 FPort = 224 Payload = 0x04121314	
15	DUT sends Unconfirmed frame	→	CMD EchoReply FPort = 224 Payload = 0x04131415	Default Rx2 Params verified
16	The TCL sends Unconfirmed frame	←	CMD RxParamSetupReq RX1DROffset = 6 RX2DataRate = 0 RX2Frequency = 869.525 Payload = 0x05X	
17	DUT sends Unconfirmed frame	→	CMD RxParamSetupAns Payload is NOT = 0x0507	DUT confirms that the parameters were not set
18	The TCL sends Unconfirmed frame	←	CMD RxParamSetupReq RX2DataRate = 9 RX1DROffset = 0 RX2Frequency = 869.525 Payload = 0x05X	
19	DUT sends Unconfirmed frame	→	CMD RxParamSetupAns Payload is NOT = 0x0507	DUT confirms that the parameters were not set
20	The TCL sends Unconfirmed frame	←	CMD RxParamSetupReq RX2Frequency = 911 MHz RX1DROffset = 0 RX2DataRate = 0 Payload = 0x05X	
21	DUT sends Unconfirmed frame	→	CMD RxParamSetupAns Payload is NOT = 0x0507	DUT confirms that the parameters were not set
22	The TCL sends Unconfirmed frame on RX1 window	←	CMD EchoReq FPort = 224 Payload = 0x04010203	
23	DUT sends Unconfirmed frame	→	CMD EchoReply FPort = 224 Payload = 0x04020304	Default Rx1 Params verified
24	The TCL sends Unconfirmed frame on RX2 window	←	CMD EchoReq FPort = 224 Payload = 0x04121314	

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
25	DUT sends Unconfirmed frame	→	CMD EchoReply FPort = 224 Payload = 0x04131415	Default Rx2 Params verified

2.5.5. RXTimingSetupReq

The **TCL** sends a MAC command to change the timing of the reception windows. The **DUT** must reply with 0x08 (and no *RXTimingSetupAns* payload). The **TCL** tests RX1 and RX2 downlink windows using the new parameters.

Finally, standard settings are applied and tested again with echo command.

2.5.5.a. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame FCntUp = y	→	FPort = 224	
2	The TCL sends Unconfirmed frame	←	CMD RxTimingSetupReq Payload = 0x08X Delay (i) = [1-15]	
3	DUT sends Unconfirmed frame <i>Repeat 3 times until a downlink is received confirming the receipt of the RxTimingSetupAns</i> FCntUp ≥ y + n	→	CMD RxTimingSetupAns Payload = 0x08	
4	The TCL sends Unconfirmed frame on RX1 window	←	CMD EchoReq FPort = 224 Payload = 0x04010203 TxDelay = (i) seconds	
5	DUT sends Unconfirmed frame	→	CMD EchoReply FPort = 224 Payload = 0x04020304	Tested for new parameters on RX1
6	The TCL sends Unconfirmed frame on RX2 window	←	CMD EchoReq FPort = 224 Payload = 0x04121314 TxDelay = (i + 1) seconds	
7	DUT sends Unconfirmed frame	→	CMD EchoReply FPort = 224 Payload = 0x04131415	Tested for new parameters on RX2
8	The TCL sends Unconfirmed frame	←	CMD RxTimingSetupReq Payload = 0x08 TXDelay = 1	
9	DUT sends Unconfirmed frame	→	CMD RxTimingSetupAns Payload = 0x08	Restored to default settings
10	The TCL sends Unconfirmed frame on RX1 window	←	CMD EchoReq FPort = 224 Payload = 0x04010203 TxDelay = 1 sec	
11	DUT sends Unconfirmed frame	→	CMD EchoReply FPort = 224 Payload = 0x04020304	Tested for default settings on RX1
12	The TCL sends Unconfirmed frame on RX2 window	←	CMD EchoReq FPort = 224 Payload = 0x04121314 TxDelay = 2 seconds	
13	DUT sends Unconfirmed frame	→	CMD EchoReply FPort = 224 Payload = 0x04131415	Tested for default settings on RX2

2.5.6. TxParamSetupReq MAC command

Since the TxParamSetupReq command is not applicable for EU region, the **DUT** must continue normal operation after receiving the TxParamSetupReq command hence ignoring or rejecting the command.

2.5.6.a. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame	→	FPort = 224	
2	The TCL sends Unconfirmed frame	←	CMD TxParamSetupReq Payload = 0x09X UplinkDwellTime = 0	
3	DUT sends Unconfirmed frame	→	FPort = 224	No response to the command but DUT continues normal operation

2.5.7. LinkCheckReq tests

After the **TCL** triggers a LinkCheckReq, the **DUT** must send a LinkCheckReq message. The **TCL** responds with a LinkCheckAns message.

Verify

- After receiving the LinkCheckAns, the **DUT** continues normal operation and sends uplinks.

2.5.7.a. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	The TCL sends Unconfirmed frame	←	CMD LinkCheckReq FPort = 224 Payload = 0x05	
2	DUT sends Unconfirmed frame	→	CMD LinkCheckReq Payload = 0x02	
3	The TCL sends Unconfirmed frame	←	CMD LinkCheckAns Payload = 0x02X	
4	DUT sends Unconfirmed frame	→	FPort = 224	
5	The TCL sends Unconfirmed frame	←	CMD Echo Req FPort 224 Payload 0x04010203	
6	DUT sends Unconfirmed frame	→	CMD Echo Reply FPort 224 Payload 0x04020304	DUT responds normally after LinkCheck

2.5.8. LinkADRReq

The following tests validate each aspect of the *LinkADRReq* command. It will be noted in the certification registration system whether the **DUT** supports the optional adaptive data rate feature. If it does, the **DUT** must pass the following tests which are depicted in Sequence Chart(s) **Error!**
Reference source not found.. 3.8.1

2.5.8.a. ADR Bit

Verify all uplink frames have the ADR bit set to 1 while the certification application is active.

2.5.8.a.i. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame FCntUp = X	→	FCtrl ADR bit = true FPort 224 Payload = RxCnt	Verify ADR bit is always set

2.5.8.b. TxPower

This test validates the **DUT** handles the Tx output power indicated in the *LinkADRReq* command as an upper-limit (i.e. allowed maximum).

The **TCL** separately tests *LinkADRReq* commands with Tx power within and outside the range specified in [2]. As part of testing values within the valid range, the **TCL** will command the device to its minimum and maximum allowable Tx power levels.

Verify

- The **DUT** responds with a successful *LinkADRAAns* for those commands whose Tx power is within the specified range. If the difference between the **DUT**'s maximum and minimum Tx output power is more than 8 dB, the **TCL** will monitor the RSSI reported by the gateway and verify a minimum difference of at least 6 dB between these settings.
- Commands whose Tx power is outside the range, the **DUT** responds with an unsuccessful *LinkADRAAns*.

2.5.8.b.i. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame	→	Payload = RxCnt FPort = 224	
2	The TCL sends Unconfirmed frame	←	CMD LinkADRReq TxPwrIndx = 7 (Minimum) DataRate = [0-5] ChnMskCntl = 0 ChnMsk = 0x0007 NbTrans = 1 Payload = 0x03X	
3	DUT sends Unconfirmed frame	→	CMD LinkADRAns Pwr ACK = true DataRate ACK = true ChnMsk ACK = true Payload = 0x0307	Command accepted
4	DUT sends Unconfirmed frame Get the RSSI value	→	Payload = RxCnt + 1 FPort = 224	
5	DUT sends Unconfirmed frame Get the RSSI value	→	Payload = RxCnt + 1 FPort = 224	
6	DUT sends Unconfirmed frame Get the RSSI value	→	Payload = RxCnt + 1 FPort = 224	
7	Calculate average value of the last 3 packets RSSI value. Average must be between -80 and -10 dB			Avg RSSI checked
8	The TCL sends Unconfirmed frame	←	CMD LinkADRReq TxPwrIndx = 0 (Maximum) DataRate = [0-5] ChnMskCntl = 0 ChnMsk = 0x0007 NbTrans = 1 Payload = 0x03X	
9	DUT sends Unconfirmed frame	→	CMD LinkADRAns Pwr ACK = true DataRate ACK = true ChnMsk ACK = true Payload = 0x0307	Command accepted
10	DUT sends Unconfirmed frame Get the RSSI value	→	Payload = RxCnt + 2 FPort = 224	
11	DUT sends Unconfirmed frame Get the RSSI value	→	Payload = RxCnt + 2 FPort = 224	
12	DUT sends Unconfirmed frame Get the RSSI value	→	Payload = RxCnt + 2 FPort = 224	

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
13	Calculate average value of the last 3 packets RSSI value. Average must be between -80 and -10 dB Avg RSSI for Max. Tx Power must be at least 6dB higher than Avg. RSSI for Min. Tx Power (applicable if difference between DUT's max and min Tx Power > 8dB)			Avg RSSI checked
14	The TCL sends Unconfirmed frame	←	CMD LinkADRReq TxPwrIndx = 8 DataRate = [0-5] ChnMskCntl = 0 ChnMsk = 0x0007 NbTrans = 1 Payload = 0x03X	
15	DUT sends Unconfirmed frame	→	CMD LinkADRAns Pwr ACK = false DataRate ACK = true ChnMsk ACK = true Payload = 0x0303	Command rejected

2.5.8.c. Uplink Channel Management

This test validates the **DUT**'s ability to process commands to operate at the required data rates as specified in [2]document.

2.5.8.c.i. Required data rates

The **TCL** sends MAC commands to change the **DUT** to each of the required data rates DR0 - DR5 (SF12BW125 - SF7BW125). The **DUT** must reply to each request with a successful **LinkADRAns**. The **TCL** verifies that the uplink data rate is as requested. The commands use channel mask index 0 and a channel mask value of 0x0007.

Verify

- The **DUT** replies to each valid request with a successful **LinkADRAns**.
- The data rate of subsequent uplinks is as commanded.

2.5.8.c.i.1. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame FCntUp = n	→	FPort = 224	
2	The TCL sends Unconfirmed frame	←	CMD LinkADRReq TxPwrIndx = 0 DataRate = 0 ChnMskCntl = 0 ChnMsk = 0x0007 NbTrans = 1 Payload = 0x03X	
3	DUT sends Unconfirmed frame FCntUp = n + 1	→	CMD LinkADRAns DataRate = 0 Payload = 0x0307	DR0 tested
4	DUT sends Unconfirmed frame FCntUp = i	→	FPort = 224	
5	The TCL sends Unconfirmed frame	←	CMD LinkADRReq TxPwrIndx = 0 DataRate = 1 ChnMskCntl = 0 ChnMsk = 0x0007 NbTrans = 1 Payload = 0x03X	
6	DUT sends Unconfirmed frame FCntUp = i + 1	→	CMD LinkADRAns DataRate = 1 Payload = 0x0307	DR1 tested
7	DUT sends Unconfirmed frame FCntUp = j	→	FPort = 224	
8	The TCL sends Unconfirmed frame	←	CMD LinkADRReq TxPwrIndx = 0 DataRate = 2 ChnMskCntl = 0 ChnMsk = 0x0007 NbTrans = 1 Payload = 0x03X	
9	DUT sends Unconfirmed frame FCntUp = j + 1	→	CMD LinkADRAns DataRate = 2 Payload = 0x0307	DR2 tested
10	DUT sends Unconfirmed frame FCntUp = k	→	FPort = 224	
11	The TCL sends Unconfirmed frame	←	CMD LinkADRReq TxPwrIndx = 0 DataRate = 3 ChnMskCntl = 0 ChnMsk = 0x0007 NbTrans = 1 Payload = 0x03X	
12	DUT sends Unconfirmed frame FCntUp = k + 1	→	CMD LinkADRAns DataRate = 3 Payload = 0x0307	DR3 tested
13	DUT sends Unconfirmed frame FCntUp = l	→	FPort = 224	

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
14	The TCL sends Unconfirmed frame	←	CMD LinkADRReq TxPwrIndx = 0 DataRate = 4 ChnMskCntl = 0 ChnMsk = 0x0007 NbTrans = 1 Payload = 0x03X	
15	DUT sends Unconfirmed frame FCntUp = l + 1	→	CMD LinkADRAns DataRate = 4 Payload = 0x0307	DR4 tested
16	DUT sends Unconfirmed frame FCntUp = m	→	FPort = 224	
17	The TCL sends Unconfirmed frame	←	CMD LinkADRReq TxPwrIndx = 0 DataRate = 5 ChnMskCntl = 0 ChnMsk = 0x0007 NbTrans = 1 Payload = 0x03X	
18	DUT sends Unconfirmed frame FCntUp = m + 1	→	CMD LinkADRAns DataRate = 5 Payload = 0x0307	DR 5 tested

2.5.8.c.ii. Optional data rates

This test is validated if the **DUT** declares it's support for the optional data rate – DR6 (SF7BW250) and DR7 (FSK50) in the Certification Registration Application.

The **TCL** sends MAC commands to change the **DUT** to the optional data rates. The **DUT** must reply with a successful *LinkADRAns* and the uplink data rate must match the requested data rate.

Verify

- The **DUT** replies to each valid request with a successful *LinkADRAns*.
- The data rate of subsequent uplinks is as commanded.

2.5.8.c.ii.1. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame FCntUp = j	→	FPort = 224	
2	The TCL sends Unconfirmed frame	←	CMD NewChannelReq ChannelIndex = 3 Freq = 868.1 MHz DRRange = 60 Payload = 0x07X Note: If DR6 is not supported, steps 2 to 6 must be skipped	
3	DUT sends Unconfirmed frame FCntUp = j + 1	→	CMD NewChannelAns Payload = 0x0703	Optional channel for DR6 added
4	DUT sends Unconfirmed frame FCntUp = j + 2	→	FPort = 224	
5	The TCL sends Unconfirmed frame	←	CMD LinkADRReq TxPwrIndx = 0 DataRate = 6 ChnMskCntl = 0 ChnMsk = 0x000F NbTrans = 1 Payload = 0x03X	
6	DUT sends Unconfirmed frame FCntUp = j + 3	→	CMD LinkADRAns DataRate = 6 Payload = 0x0307	DR6 tested
7	The TCL sends Unconfirmed frame	←	CMD NewChannelReq ChannelIndex = 3 Freq = 868.1 MHz DRRange = 70 Payload = 0x07X Note: If DR7 is not supported, steps 7 to 10 must be skipped	
8	DUT sends Unconfirmed frame FCntUp = k	→	CMD NewChannelAns Payload = 0x0703	Optional channel for DR7 added
9	The TCL sends Unconfirmed frame	←	CMD LinkADRReq TxPwrIndx = 0 DataRate = 7 ChnMskCntl = 0 ChnMsk = 0x000F NbTrans = 1 Payload = 0x03X	
10	DUT sends Unconfirmed frame FCntUp = k + 1	→	CMD LinkADRAns DataRate = 7 Payload = 0x0307	DR7 tested

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
11	The TCL sends Unconfirmed frame	←	CMD LinkADRReq TXPower = 0 DataRate = 0 ChMaskCntl = 0 ChMask = 0x0007 NbTrans = 1 Payload = 0x03X	
12	DUT sends Unconfirmed frame	→	CMD LinkADRAns Payload = 0x0307	DUT reverted to default settings
13	The TCL sends Unconfirmed frame	←	CMD NewChannelReq ChIndex = 3 Freq = 0 MHz Payload = 0x07X	
14	DUT sends Unconfirmed frame	→	CMD NewChannelAns Payload = 0x0703	Removed additional channel added

2.5.8.c.iii. Unsupported data rates

The **TCL** sends MAC commands to change the **DUT** to an unsupported data rate.

The **DUT** must reply with an unsuccessful *LinkADRAns* and the uplink data rate must not change.

Verify

- The **DUT** replies to each valid request with an unsuccessful *LinkADRAns*.
- The uplink data rate does not change.

2.5.8.c.iii.1. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame FCntUp = n	→	FPort = 224	
2	The TCL sends Unconfirmed frame	←	CMD LinkADRRReqTxPwrIndx = 0 DataRate = 8 ChnMskCntl = 0 ChnMsk = 0x0007 NbTrans = 1 Payload = 0x03X	
3	DUT sends Unconfirmed frame FCntUp = n + 1	→	CMD LinkADRAns Payload is NOT = 0x0307	Request rejected for unsupported DR
4	DUT sends Unconfirmed frame FCntUp = n + 2	→	FPort = 224	
5	The TCL sends Unconfirmed frame	←	CMD LinkADRRReq TxPwrIndx = 0 DataRate = 6 ChnMskCntl = 0 ChnMsk = 0x0007 NbTrans = 1 Payload = 0x03X	
6	DUT sends Unconfirmed frame FCntUp = n + 3	→	CMD LinkADRAns Payload is NOT = 0x0307	Request rejected, as DR 6 is not allowed for the default channels
7	DUT sends Unconfirmed frame FCntUp = n + 4	→	FPort = 224	
8	The TCL sends Unconfirmed frame	←	CMD LinkADRRReq TxPwrIndx = 0 DataRate = 7 ChnMskCntl = 0 ChnMsk = 0x0007 NbTrans = 1 Payload = 0x03X	
9	DUT sends Unconfirmed frame FCntUp = n + 5	→	CMD LinkADRAns Payload is NOT = 0x0307	Request rejected, as DR 7 is not allowed for the default channels

2.5.8.c.iv. ChannelMask functionality

The **TCL** sends *NewChannelReq* command to configure a new channel and *LinkADRRReq* to disable that channel using the channel mask. The **TCL** verifies that no transmissions are sent on the new channel. The **TCL** then sends a *LinkADRRReq* to enable the new

channel. The **TCL** verifies that the **DUT** begins using the new channel in a random sequence.

Finally, the additional channel is removed.

2.5.8.c.v. Disable all Channels

Lastly, the **TCL** tries to disable all channels by sending a *LinkADRReq* command with channel mask control index of 0 and a channel mask of 0x0000. The **DUT** must reply with an unsuccessful *LinkADRAns*, (i.e. not 0x0307).

The **TCL** verifies that **DUT** still uses the default uplink channels in random sequence at an appropriate data rate.

2.5.8.c.v.1. Test Procedure Message Sequence Chart for Channel Mask and Disable all Channels

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame FCntUp = n	→	FPort = 224	
2	The TCL sends Unconfirmed frame	←	CMD NewChannelReq ChannelIndex = 3 Freq = 867.7 MHz DRRange = 50 Payload = 0x07X CMD LinkADRReq TxPwrIndx = 0 DataRate = 0 ChnMskCntl = 0 ChnMsk = 0x0007 NbTrans = 1 Payload = 0x03X	Adds new channel and disables it
3	DUT sends Unconfirmed frame FCntUp = n + 1	→	DataRate = Z CMD NewChannelAns Payload = 0x0703 CMD LinkADRAns Payload = 0x0307	
4	Wait until all channels configured are used at least once to confirm the channel plan [Wait for a maximum of (5 * number of channels configured) uplink packets]	→	FPort = 224	DUT does not modify its channel plan
5	DUT sends Unconfirmed frame FCntUp = i	→	FPort = 224	
6	The TCL sends Unconfirmed frame	←	CMD LinkADRReq TxPwrIndx = 0 DataRate = 0 ChnMskCntl = 0 ChnMsk = 0x000F NbTrans = 1 Payload = 0x03X	Enables the new channel
7	DUT sends Unconfirmed frame FCntUp = i + 1	→	DataRate = 0 CMD LinkADRAns Payload = 0x0307	
8	Wait until all channels configured are used at least once to confirm the channel plan [Wait for a maximum of (5 * number of channels configured) uplink packets]	→	FPort = 224	New channel added to default plan
9	DUT sends Unconfirmed frame FCntUp = j	→	FPort = 224	
10	The TCL sends Unconfirmed frame	←	CMD NewChannelReq ChannelIndex = 3 Freq = 0 MHz Payload = 0x07X	Removes new channel
11	DUT sends Unconfirmed frame FCntUp = j + 1	→	DataRate = 0 CMD NewChannelAns Payload = 0x0703	Channel removed

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
12	DUT sends Unconfirmed frame FCntUp = k	→	FPort = 224	
13	The TCL sends Unconfirmed frame	←	CMD LinkADRReq TxPwrIndx = 0 DataRate = 0 ChnMskCntl = 0 ChnMsk = 0x0000 NbTrans = 1 Payload = 0x03X	Tries to disable all channels
14	DUT sends Unconfirmed frame FCntUp = k + 1	→	DataRate = 0 CMD LinkADRAns Payload IS NOT = 0x0307	Channel plan not modified

2.5.8.d. Redundancy

This test validates the **DUT's** correct implementation *NbTrans* setting within the *LinkADRReq* command. The **TCL** sends *LinkADRReq* requesting a repetition count of 2.

Verify

- All subsequent unconfirmed uplink frames from the **DUT** are transmitted twice (the same sequence number is received twice by the **TCL**). The number of redundant uplinks verified by the harness is at the discretion of the Test Tool vendor.
- The **DUT** does not repeat the transmission if a downlink is received during the RX1 window.
- The **DUT** does not repeat the transmission if a downlink is received during the RX2 window.

Once the above criteria are verified for a *NbTrans* setting of 2, the **DUT** is commanded back to the default of 1 by the **TCL** using a *LinkADRReq* command wherein the *NbTrans* value is 0. This default setting is also validated.

The above test is repeated for *NbTrans* = 3 within the *LinkADRReq* command.

2.5.8.d.i. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame FCntUp = n	→	FPort = 224	
2	The TCL sends Unconfirmed frame	←	CMD LinkADRReq TxPwrIndx = 0 DataRate = 0 ChnMskCntl = 0 ChnMsk = 0x0007 NbTrans = 2 Payload = 0x03X	
3	DUT sends Unconfirmed frame FCntUp = n + 1	→	CMD LinkADRAns Payload = 0x0307	
4	DUT sends Unconfirmed frame FCntUp = n + 1	→	CMD LinkADRAns Payload = 0x0307	Uplink sent twice
5	DUT sends Unconfirmed frame FCntUp = n + 2	→	FPort = 224 Payload = RxCnt	
6	DUT sends Unconfirmed frame FCntUp = n + 2	→	FPort = 224 Payload = RxCnt	
7	DUT sends Unconfirmed frame FCntUp = n + 3	→	FPort = 224 Payload = RxCnt	
8	The TCL sends Unconfirmed frame on RX1 window	←		
9	DUT sends Unconfirmed frame FCntUp = n + 4	→	FPort = 224 Payload = RxCnt + 1	Transmit not repeated when downlink received on RX1
10	DUT sends Unconfirmed frame FCntUp = n + 4	→	FPort = 224 Payload = RxCnt + 1	
11	DUT sends Unconfirmed frame FCntUp = n + 5	→	FPort = 224 Payload = RxCnt + 1	
12	The TCL sends Unconfirmed frame on RX2 window	←		
13	DUT sends Unconfirmed frame FCntUp = n + 6	→	FPort = 224 Payload = RxCnt + 2	Transmit not repeated when downlink received on RX2
14	The TCL sends Unconfirmed frame	←	CMD LinkADRReq TxPwrIndx = 0 DataRate = 0 ChnMskCntl = 0 ChnMsk = 0x0007 NbTrans = 1 Payload = 0x03X	
15	DUT sends Unconfirmed frame FCntUp = n + 7	→	CMD LinkADRAns Payload = 0x0307	Uplink sent once
16	DUT sends Unconfirmed frame FCntUp = n + 8	→	FPort = 224 Payload = RxCnt + 3	

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
17	The TCL sends Unconfirmed frame	←	CMD LinkADRReq TxPwrIndx = 0 DataRate = 0 ChnMskCntl = 0 ChnMsk = 0x0007 NbTrans = 3 Payload = 0x03X	
18	DUT sends Unconfirmed frame FCntUp = n + 9	→	CMD LinkADRAns Payload = 0x0307	
19	DUT sends Unconfirmed frame FCntUp = n + 9	→	CMD LinkADRAns Payload = 0x0307	
20	DUT sends Unconfirmed frame FCntUp = n + 9	→	CMD LinkADRAns Payload = 0x0307	Uplink sent thrice
21	DUT sends Unconfirmed frame FCntUp = n + 10	→	FPort = 224 Payload = RxCnt + 4	
22	The TCL sends Unconfirmed frame	←		
23	DUT sends Unconfirmed frame FCntUp = n + 11	→	FPort = 224 Payload = RxCnt + 5	Transmit not repeated when downlink received
24	DUT sends Unconfirmed frame FCntUp = n + 11	→	FPort = 224 Payload = RxCnt + 5	
25	DUT sends Unconfirmed frame FCntUp = n + 11	→	FPort = 224 Payload = RxCnt + 5	
26	The TCL sends Unconfirmed frame	←	CMD LinkADRReq TxPwrIndx = 0 DataRate = 0 ChnMskCntl = 0 ChnMsk = 0x0007 NbTrans = 0 Payload = 0x03X	
27	DUT sends Unconfirmed frame FCntUp = n + 12	→	CMD LinkADRAns Payload = 0x0307	Uplink sent once
28	DUT sends Unconfirmed frame FCntUp = n + 13	→	FPort = 224 Payload = RxCnt + 6	

2.5.8.e. Data Rate Decay

This test validates the **DUT**'s implementation of the *ADR_ACK_LIMIT*, *ADR_ACK_DELAY*, and use of the *ADRACKReq* bit in its uplinks' frame header. A tolerance of +/- 2 uplinks is allowed for the **DUT**'s implementation of both the *ADR_ACK_LIMIT* and *ADR_ACK_DELAY* parameters. A default DR of DR0 is assumed.

The **TCL** sends *LinkADRReq* to configure the **DUT** to use *SF7BW125 (DR5)*. It then stops sending downlinks until the **DUT** decays to *SF12BW125 (DR0)*. Verify

- After sending *ADR_ACK_LIMIT* (64 +/- 2) uplinks the **DUT** must - in a sustained absence of downlinks - set the *ADRACKReq* bit in the next *ADR_ACK_DELAY* (32 +/- 2) uplinks' frame headers.

- As of its 96th (+/- 4) consecutive uplink in the continued absence of downlinks, the **DUT** must lower its data rate and leave the *ADRACKReq* bit set so long as the new data rate is greater than its default DR.
- The **DUT** further lowers its data rate by 1 each time another *ADR_ACK_DELAY* cycle occurs (32 +/- 2) uplinks are sent in the sustained absence of downlinks.

2.5.8.e.i. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame FCntUp = n	→	FPort = 224	
2	The TCL sends Unconfirmed frame	←	CMD LinkADRReq TxPwrIndx = 0 DataRate = 5 ChnMskCntl = 0 ChnMsk = 0x0007 NbTrans = 1 Payload = 0x03X	
3	DUT sends Unconfirmed frame FCntUp = n + 1	→	CMD LinkADRAns DataRate = 5 Payload = 0x0307	
4	DUT sends Unconfirmed frame FCntUp = n + 2 Repeat 63 times (+/- 2) without receiving any downlinks	→	FPort = 224 DataRate = 5 ADRAckReq = False	
5	DUT sends Unconfirmed frame FCntUp = n + 65 (+/- 2) Repeat 32 times (+/- 2) without receiving any downlinks (starting with n + 65 (+/- 2))	→	FCtrl ADRAckReq = True DataRate = 5 FPort = 224	
6	DUT sends Unconfirmed frame FCntUp = n + 97 (+/- 4) Repeat 32 times (+/- 2) without receiving any downlinks	→	FCtrl ADRAckReq = True DataRate = 4 FPort = 224	DUT switches to DR4
7	DUT sends Unconfirmed frame FCntUp = n + 129 (+/- 6) Repeat 32 times (+/- 2) without receiving any downlinks	→	FCtrl ADRAckReq = True DataRate = 3 FPort = 224	DUT switches to DR3
8	DUT sends Unconfirmed frame FCntUp = n + 161 (+/- 8) Repeat 32 times (+/- 2) without receiving any downlinks	→	FCtrl ADRAckReq = True DataRate = 2 FPort = 224	DUT switches to DR2
9	DUT sends Unconfirmed frame FCntUp = n + 193 (+/- 10) Repeat 32 times (+/- 2) without receiving any downlinks	→	FCtrl ADRAckReq = True DataRate = 1 FPort = 224	DUT switches to DR1
10	DUT sends Unconfirmed frame FCntUp = n + 225 (+/- 12)	→	FCtrl ADRAckReq = False DataRate = 0 FPort = 224	DUT switches to DR0

2.5.8.f. Command Block Channel Management

This test validates the **DUT** correctly processes blocks of multiple *LinkADRReq* commands included in the same downlink. The **DUT** is expected to service each of these MAC commands in the same sequence as it is ordered in the FOpts or FrmPayload field.

2.5.8.f.i. Successful LinkADRReq block

The **TCL** sends multiple *LinkADRReq* commands in a single downlink as shown in **Table 2.5-1**. The first command tries to disable all channels. The channel mask the second command enables is *channel 0*, and the data rate, DR4, is appropriate for this channel. The last command enables all defined channels using the channel mask control 6 and configures the data rate DR3. The **DUT** is expected to respond with *LinkADRAAns 0x0307* for all *LinkADRReq* commands.

LinkADRReq CMD Index	DR	TxPower	Channel Mask	MaskCntl	NbTrans
1	0	1	0x0000	0	0
2	4	4	0x0001	0	0
3	3	1	0x0000	6	1

Table 2.5-1: Successful LinkADRReq block

Note:

- The DR, TX Power, and NbTrans values of only the last command are implemented by the **DUT**. Values for these parameters in any preceding command must be ignored.
- After processing the command block shown above, the **DUT** is expected to be configured as follows:

Channel Plan: Channels 0 – 2

Data Rate: DR3

TxPower: MaxEIRP - 2dB

Verify

- The **DUT** replies with an uplink containing a successful *LinkADRAAns* for each command in the block.
- The uplink containing the answers and all subsequent transmissions are sent using the default channels at the commanded DR.

2.5.8.f.i.1. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame FCntUp = n	→	FPort = 224	
2	The TCL sends Unconfirmed frame	←	CMD1 LinkADRRReq TxPwrIndx = 1 DataRate = 0 ChnMskCntl = 0 ChnMsk = 0x0000 NbTrans = 0 Payload = 0x03X CMD2 LinkADRRReq TxPwrIndx = 4 DataRate = 4 ChnMskCntl = 0 ChnMsk = 0x0001 NbTrans = 0 Payload = 0x03X CMD3 LinkADRRReq TxPwrIndx = 1 DataRate = 3 ChnMskCntl = 6 ChnMsk = 0x0000 NbTrans = 1 Payload = 0x03X	
3	DUT sends Unconfirmed frame FCntUp = n + 1	→	DataRate = 3 CMD1 LinkADRAns Payload = 0x0307 CMD2 LinkADRAns Payload = 0x0307 CMD3 LinkADRAns Payload = 0x0307	Uplinks @DR3, 125KHz, Channels 0- 2 used
4	Wait for a maximum of 5 * 3 (number of channels configured) = 15 uplink packets to be sent, i.e. until all default channels are used at least once.	→	FPort = 224	All channels must be used at least once

2.5.8.f.ii. Unsuccessful LinkADRRReq block

The **TCL** sends several *LinkADRRReq* commands in a single downlink as shown in **Table 2.5-2**. In this case the last command requires all channels to be disabled, all commands should be rejected.

LinkADRRReq CMD Index	DR	TxPower	Channel Mask	MaskCntl	NbTrans
1	4	4	0x0001	0	0

2	3	1	0x0000	6	1
3	0	1	0x0000	0	1

Table 2.5-2: Unsuccessful LinkADRRReq block

Verify

- The **DUT** replies with an uplink containing an unsuccessful LinkADRAns for each command in the block.
- The uplink data rate does not change and the **DUT** continues using the three default channels at the previously configured data rate.

2.5.8.f.ii.1. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame FCntUp = n	→	FPort = 224 DataRate = X	
2	The TCL sends Unconfirmed frame	←	CMD1 LinkADRRReq TxPwrIndx = 4 DataRate = 4 ChnMskCntl = 0 ChnMsk = 0x0001 NbTrans = 0 Payload = 0x03X CMD2 LinkADRRReq TxPwrIndx = 1 DataRate = 3 ChnMskCntl = 6 ChnMsk = 0x0000 NbTrans = 1 Payload = 0x03X CMD3 LinkADRRReq TxPwrIndx = 1 DataRate = 0 ChnMskCntl = 0 ChnMsk = 0x0000 NbTrans = 1 Payload = 0x03X	
3	DUT sends Unconfirmed frame FCntUp = n + 1	→	DataRate = X CMD1 LinkADRAns Payload is NOT = 0x0307 CMD2 LinkADRAns Payload is NOT = 0x0307 CMD3 LinkADRAns Payload is NOT = 0x0307	All commands are rejected as the last command requires all channels to be disabled

2.5.9. RX Window test

2.5.9.a. RX1 Receive Window Test

This test validates the **DUT**'s capability to receive data on RX1 for data rates as specified in [2]

TX-DR to RX1-DR Mapping (Per RX1DR Offset Index)						
	RX1 DR Offset Index					
Operating Uplink DR	0	1	2	3	4	5
DR0	DR0	DR0	DR0	DR0	DR0	DR0
DR1	DR1	DR0	DR0	DR0	DR0	DR0
DR2	DR2	DR1	DR0	DR0	DR0	DR0
DR3	DR3	DR2	DR1	DR0	DR0	DR0
DR4	DR4	DR3	DR2	DR1	DR0	DR0
DR5	DR5	DR4	DR3	DR2	DR1	DR0
DR6 (Optional)	DR6	DR5	DR4	DR3	DR2	DR1
DR7 (Optional)	DR7	DR6	DR5	DR4	DR3	DR2

Table 2.5-3: TX-DR to RX1-DR Mapping Per Offset

For each combination of Operating Uplink DR to RX1 offset in the above table:

- The **TCL** commands the **DUT** to implement the target configuration and subsequently validates corresponding *LinkADRs* and *RXParamSetupAns* from the **DUT** indicating it has implemented the target configuration.
- The **TCL** then sends 5 consecutive downlinks to the **DUT** on the RX1 window where:
 - At least one of the frames is the maximum allowed payload for the current RX1 DR (with respect to the current offset).
 - At least one of the frames is confirmed, requiring the **DUT** to set the ACK bit in the subsequent uplink.
- The **TCL** validates reception by verifying for each downlink that the subsequent uplink contains a payload indicating the **DUT**'s downlink counter has incremented by one. The uplink following the confirmed downlink must have its ACK bit set to true as well.

For the given RX1 Offset, the **TCL** then transitions the **DUT** through the remaining Uplink DR settings, verifying the respective *LinkADRs* and repeating the above procedure for each.

Once the combinations for the current RX1 Offset are exhausted, the **TCL** commands the **DUT** to the next RX1 Offset verifies the *RXParamSetupAns* and repeats the same series of downlink tests. The **TCL** proceeds in this fashion exhausting all combinations.

For each unique RX1 DR shown in **Error! Reference source not found.**, the **TCL** will perform the **Sufficient Reception** test defined in 2.4.1.c Downlink Error Rate. For these tests RX1 DR Offset will be set to 0. The **TCL** sends 60 downlink packets in the RX1 window and verifies that at least 57 messages were received by the **DUT**.

2.5.9.a.i. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame	→	FPort = 224	
2	The TCL sends Unconfirmed frame	←	CMD NewChannelReq If DR6 is supported ChannelIndex = 3 Frequency = 868.1 MHz DRRange = 0x66 If DR7 is supported ChannelIndex = 4 Frequency = 868.1 MHz DRRange = 0x77 Payload = 0x07X Note: If DR6 and 7 are not supported, skip steps 2 and 3	
3	DUT sends Unconfirmed frame	→	CMD NewChannelReq Payload = 0x07030x0703	Added new channel
4	The TCL sends Unconfirmed frame	←	CMD LinkADRReq DataRate = X, where X = [0-7] (as supported) Payload = 0x03X CMD RxParamSetupReq Rx1DROffset = [0-5] Rx2DataRate = 0 Rx2 Frequency = 869.525. MHz Payload = 0x05X	
5	DUT sends Unconfirmed frame	→	CMD LinkADRAns Payload = 0x0307 CMD RxParamSetupAns Payload = 0x0507 Payload (RxCnt) = x (May be a separate uplink if MAC commands are sent in Port 0)	
6	The TCL sends Unconfirmed frame on RX1 window <i>Repeat 4 times</i>	←	DR – X Payload = 0x03	
7	DUT sends Unconfirmed frame (4 times)	→	Payload (RxCnt) ≤ x + 4	
8	The TCL sends a Confirmed frame on RX1 window	←	DR – X Payload = Max allowed payload as defined in Maximum Payload size table in [2]	

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
9	DUT sends Unconfirmed frame	→	ACK Bit = True (else, a separate Unconfirmed frame with ACK Bit = True with empty port is allowed before this step) Payload (RxCnt) <= x + 5	
	<i>For each RX1DROffset combination in the RX1DROffset table in [2], the TCL transitions the DUT to each Uplink Data Rate by repeating Steps 4-9.</i>			
10	DUT sends Unconfirmed frame	→	FPort = 224	
11	The TCL sends Unconfirmed frame	←	CMD LinkADRReq DataRate = X, where X = [0-7] (as supported) CMD RxParamSetupReq Rx1DROffset = 0 Payload = 0x05X	
12	DUT sends Unconfirmed frame	→	CMD LinkADRAns Payload = 0x0307 CMD RxParamSetupAns Payload = 0x0507 Payload (RxCnt) = y (May be a separate uplink if MAC commands are sent in Port 0)	
13	The TCL sends Unconfirmed frame on RX1 window <i>Repeat 60 times</i>	←	DataRate = X Payload = 0x03 (MaxLen for RX1 DR)	
14	DUT sends Unconfirmed frame	→	Payload (RxCnt) <= y + 60 Payload (RxCnt) >= y + 57	Reception failure rate <=5%
	<i>For each unique RX1 Data Rate in the RX1DROffset table in [2], the TCL will perform the Sufficient Reception test by repeating Steps 10-14.</i>			
15	The TCL sends Unconfirmed frame	←	CMD LinkADRReq DataRate = 5 Payload = 0x03X	
16	DUT sends Unconfirmed frame	→	CMD LinkADRAns Payload = 0x0307	

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
17	The TCL sends Unconfirmed frame	←	CMD NewChannelReq ChannelIndex = 3 Frequency = 0 ChannelIndex = 4 Frequency = 0 Payload = 0x07X	
18	DUT sends Unconfirmed frame	→	CMD NewChannelReq Payload = 0x07030x0703	Removed channel added

2.5.9.b. RX2 Receive Window Test

This test validates the **DUT**'s capability to receive data on RX2 for all data rates.

For each unique RX2 DR, the **TCL** will perform the **Sufficient Reception** test defined in 2.4.1.c Downlink Error Rate. For these tests RX1 DR Offset will be set to 0. The **TCL** sends 60 downlink packets in the RX2 window and verifies that at least 57 messages were received by the **DUT**.

2.5.9.b.i. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame	→	FPort = 224	
2	The TCL sends Unconfirmed frame	←	CMD RxParamSetupReq Rx1DROffset = 0 Rx2DataRate = X, where X = 1-7] (as supported) Rx2 Frequency = 869.525. MHz Payload = 0x05X	
3	DUT sends Unconfirmed frame	→	CMD RxParamSetupAns Payload = 0x0507 Payload (RxCnt) = y (May be a separate uplink if MAC commands are sent in Port 0)	
4	The TCL sends Unconfirmed frame on RX2 window Repeat 60 times	←	DataRate = X Payload = 0x03 (MaxLen for RX2 DR)	
5	DUT sends Unconfirmed frame	→	Payload (RxCnt) <= y + 60 Payload (RxCnt) >= y + 57	Reception failure rate <=5%
	For each RX2DataRate [0-7] supported by the DUT, the TCL will perform the Sufficient Reception test by repeating Steps 2-5.			

2.5.9.c. **RX1 and RX2 simultaneous frames**

This test validates that when **TCL** sends frames on both RX1 and RX2 windows simultaneously, **DUT** responds to the RX1 frame but does not open the RX2 window.

2.5.9.c.i. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame	→	FPort = 224 Payload (RxCnt) = x	
2	The TCL sends Unconfirmed frame	←	CMD LinkADRReq DataRate = 2 Payload = 0x03X	
3	DUT sends Unconfirmed frame	→	CMD LinkADRAns DataRate = 2 Payload = 0x0307	
4	The TCL sends Unconfirmed frame on RX1 window	←	FPort = 224 Payload 0x03	
5	The TCL sends Unconfirmed frame on RX2 window	←	FPort 224 Payload 0x03	
6	DUT sends Unconfirmed frame	→	FPort = 224 Payload (RxCnt) = x + 1	DUT does not open RX2 Window if it received downlink in RX1 Note: RxCnt is incremented only for applicative downlinks

2.5.9.d. **RX Oversized Payload**

A follow-up negative test must be performed for each oversized scenario shown in the table below. After commanding the **DUT** such that the target RX DR is achieved, the TCL sends a downlink whose payload is one byte greater than the scenario's maximum. Note the payload content is random (i.e. not the echo command).

The TCL should use both confirmed and unconfirmed oversized frames; as well as target both receive windows.

RX DR	Max Downlink Payload (N)	Oversized Payload (N+1)
DR0	51 bytes	52 bytes
DR1	51 bytes	52 bytes
DR2	51 bytes	52 bytes
DR3	115 bytes	116 bytes
DR4	242 bytes	243 bytes*

DR5	242 bytes	243 bytes*
DR6 (optional)	242 bytes	243 bytes*
DR7 (optional)	242 bytes	243 bytes*

Table 2.5-4: RX-DR to Max Downlink and Oversized Payload

Verify

- The **DUT** continues normal operation in the presence of oversized downlinks. Uplinks following the oversized downlinks may or may not have a payload indicating the **DUT's** downlink counter has incremented. If the downlink is confirmed, then the subsequent uplink may have its ACK bit set to true or false.

2.5.9.d.i. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame	→	FPort = 224	
2	The TCL sends Unconfirmed frame	←	CMD LinkADRReq DataRate = [0-7] (as supported)	
3	DUT sends Unconfirmed frame	→	CMD LinkADRAns Payload = 0x0307	
4	The TCL sends Unconfirmed frame on RX1 window	←	FPort = 224 Payload = 0x03 (MaxLen + 1 for UL DR-X)	
5	DUT sends Unconfirmed frame	→	FPort = 224	DUT continues normal operation
6	The TCL sends Unconfirmed frame on RX2 window	←	FPort = 224 Payload = 0x03 (MaxLen + 1 for UL DR-X)	
7	DUT sends Unconfirmed frame	→	FPort = 224	DUT continues normal operation
8	The TCL sends Confirmed frame on RX1 window	←	FPort = 224 Payload = 0x03 (MaxLen + 1 for UL DR-X)	
9	DUT sends Unconfirmed frame	→	ACK = True or No ACK	DUT continues normal operation
10	The TCL sends Confirmed frame on RX2 window	←	FPort = 224 Payload = 0x03 (MaxLen + 1 for UL DR-X)	
11	DUT sends Unconfirmed frame	→	ACK = True or No ACK	DUT continues normal operation

2.5.9.e. Maximum Allowed Payload

The purpose of this test is to validate the maximum allowable payload sizes in both uplink as well as downlink transmissions. The test depends on properly functioning ADR and RXParamSetup MAC command control. The first part validates the maximum uplink payload for each of the five uplink data rates against both receive windows, testing both positive and negative scenarios. The second part validates the maximum downlink payload for the various RX1DR offsets as well as various RX2 data rates.

2.5.9.e.i. Uplink Payload

The **TCL** commands the **DUT** with some combination of the following settings and validates the device successfully implements them before proceeding to test max payload handling:

- RX1DROffset = 0
- RX2DataRate = any of DR4 – DR5

The following table is provided for procedural reference:

Uplink DR	Max Uplink Payload (N)	Oversized Payload (N+1)
DR0	51 bytes	52 bytes
DR1	51 bytes	52 bytes
DR2	51 bytes	52 bytes
DR3	115 bytes	116 bytes
DR4	242 bytes	243 bytes*
DR5	242 bytes	243 bytes*
DR6 (optional)	242 bytes	243 bytes*
DR7 (optional)	242 bytes	243 bytes*

Table 2.5-5: Max and Oversized Uplink

2.5.9.e.i.1. Max Payload via Echo

These tests are performed on both RX1 and Rx2 Windows. For each 'Uplink DR', the **TCL** first commands the **DUT** to use the target 'Uplink DR' and verifies the configuration. It then sends an echo command (0x04) whose payload size is the current maximum allowed for the uplink DR as defined under the 'Max Uplink Payload (N)' column of **Error! Reference source not found.**

Verify

- The **DUT** responds to each echo command with an echo answer.
- The payload size is in fact the maximum for the **DUT**'s current DR and its content is correct.

2.5.9.e.i.1.1. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame	→	FPort = 224	
2	The TCL sends Unconfirmed frame	←	CMD LinkADRReq DataRate = X in [0-7] (as supported) Payload = 0x03X CMD RxParamSetupReq Rx1DROffset = 0 Rx2DataRate = [4-5] Frequency = 869.525 MHz Payload = 0x05X	
3	DUT sends Unconfirmed frame	→	CMD LinkADRAns Payload = 0x0307 CMD RxParamSetupAns LinkADRAns Payload = 0x0507 DataRate = X	
4	The TCL sends Unconfirmed frame on RX1 window	←	CMD EchoReq FPort = 224 EchoReq (i) = MaxLen for UL DR-X	
5	DUT sends Unconfirmed frame	→	DataRate = X FPort = 224 EchoReply (i') = MaxLen for DR-X	DUT echos MaxLen PDU for each Uplink DR on RX1
6	The TCL sends Unconfirmed frame on RX2 window	←	CMD EchoReq FPort = 224 EchoReq (i) = MaxLen for UL DR-X	
7	DUT sends Unconfirmed frame	→	DataRate = X FPort = 224 EchoReply (i') = MaxLen for DR-X	DUT echos MaxLen PDU for each Uplink DR on RX2

2.5.9.e.i.2. Oversized Payload via Echo

These tests are performed on the RX2 Window only. For each 'Uplink DR', the **TCL** first commands the **DUT** to use the target 'Uplink DR' and verifies the configuration. It then sends an echo command (0x04) whose payload size is one byte greater than the current maximum allowed for the uplink DR as defined under the 'Oversized Payload (N+1)' column of **Error! Reference source not found.**

* If the **TCL** supports downlinking oversized PDUs greater than 242 bytes, then this test can be performed for uplink data rates 4-7. Otherwise these (*) scenarios can be skipped.

Verify

- The **DUT** does not uplink an echo response for any of the echo commands due to uplink size limitation. It may either respond with an uplink that has no payload, or silently discard the echo-command.
- Subsequent to the first uplink sent after receiving the command, uplinks should resume containing only the default payload (downlink counter).

2.5.9.e.i.2.1. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame	→	Payload (RxCnt) = n FPort = 224	
2	The TCL sends Unconfirmed frame	←	CMD LinkADRReq DataRate = X in [0-7] (as supported) Payload = 0x03X CMD RxParamSetupReq Rx1DROffset = 0 Rx2DataRate = [4-5] Frequency = 869.525 MHz Payload = 0x05X	
3	DUT sends Unconfirmed frame	→	CMD LinkADRAns Payload = 0x0307 CMD RxParamSetupAns LinkADRAns Payload = 0x0507 DataRate = X Payload (RxCnt) = n + 1 (May be a separate uplink if MAC commands are sent in Port 0)	
4	The TCL sends Unconfirmed frame on RX2 window	←	CMD EchoReq FPort = 224 EchoReq (i) = MaxLen + 1 for UL DR-X	
5	DUT sends Unconfirmed frame	→	Payload does not exceed limits FPort = 224 or empty port	DUT continues normal operation in the presence of oversized downlinks

2.5.10. MAC Command(s) in App-Payload (FRMPayload)

These tests ensure a **DUT** appropriately accepts and processes or discards a downlink whose contents include one or more MAC commands in the App-Payload and/or the Frame options portion of the frame.

2.5.10.a. App-Payload Only (FPort = 0)

Perform the test listed below such that all MAC commands are sent to the **DUT** within the App-Payload (i.e. FRMPayload) portion of a single downlink. Successful completion of each test indicates the MAC Command payload is properly decrypted using the Network Session Key and processed by the **DUT**. It is up to the end-device to choose how it answers: it can choose either the FOpts field or the FRMPayload. Some devices for example may use the FRMPayload when the MAC command answers are larger than the 15 bytes limit of the FOpts field.

- DevStatusReq
- RxParamSetupReq
- RxTimingSetupReq
- LinkADDRReq

2.5.10.a.i. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame FCntUp = n	→	FPort = 224	
2	The TCL sends Unconfirmed frame	←	App-Payload CMD1 DevStatusReq CMD2 RxParamSetupReq CMD3 RxTimingSetupReq CMD4 LinkADDRReq FPort = 0	
3	DUT sends Unconfirmed frame FCntUp = n + 1	→	CMD1 DevStatusAns CMD2 RxParamSetupAns CMD3 RxTimingSetupAns CMD4 LinkADRAns	Successful completion of each command
4	The TCL sends Unconfirmed frame	←	Payload = 0x03 FPort = 224	
5	DUT sends Unconfirmed frame FCntUp = n + 2	→	FPort = 224 No MAC response	

2.5.10.b. Frame Options Only (FPort != 0)

Perform the test listed below such that all MAC commands are sent to the **DUT** within the Frame Options field (i.e. FOpts) portion of a single downlink. Successful completion of each test indicates the MAC Command payload is properly decrypted using the Application Session Key and processed by the **DUT**. It is up to the end-device to choose how it answers: it can choose

either the FOpts field or the FRMPayload. Some devices for example may use the FRMPayload when the MAC command answers are larger than the 15 bytes limit of the FOpts field.

- DevStatusReq
- RxParamSetupReq
- RxTimingSetupReq
- LinkADRReq

2.5.10.b.i. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame FCntUp = n	→	FPort = 224	
2	The TCL sends Unconfirmed frame	←	<u>Frame Options</u> CMD1 DevStatusReq CMD2 RxParamSetupReq CMD3 RxTimingSetupReq CMD4 LinkADRReq FPort != 0	
3	DUT sends Unconfirmed frame FCntUp = n + 1	→	CMD1 DevStatusAns CMD2 RxParamSetupAns CMD3 RxTimingSetupAns CMD4 LinkADRAns	Successful completion of each command
4	The TCL sends Unconfirmed frame	←	Payload = 0x03 FPort = 224	
5	DUT sends Unconfirmed frame FCntUp = n + 2	→	FPort = 224 No MAC response	

2.5.10.c. App-Payload and Frame Options

Verify the **DUT** ignores a downlink when MAC commands are simultaneously present in the App-Payload and Frame options (FOpts) fields.

2.5.10.c.i. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame FCntUp = n	→	Payload (RxCnt) = x FPort = 224	
2	The TCL sends Unconfirmed frame	←	<u>Frame Options</u> CMD1 LinkADRRReq CMD2 LinkADRRReq CMD3 RxParamSetupReq <u>App-Payload</u> CMD1' LinkADRRReq CMD2' RxTimingSetupReq CMD3' DevStatusReq	
3	DUT sends Unconfirmed frame FCntUp = n + 1	→	Payload (RxCnt) = x FPort = 224	Downlink ignored when MAC commands are present simultaneously in App-Payload and FOpts field
4	The TCL sends Confirmed frame	←	<u>Frame Options</u> CMD1 LinkADRRReq CMD2 LinkADRRReq CMD3 RxParamSetupReq <u>App-Payload</u> CMD1' LinkADRRReq CMD2' RxTimingSetupReq CMD3' DevStatusReq	
5	DUT sends Unconfirmed frame FCntUp = n + 2	→	Payload (RxCnt) = x FPort = 224	Downlink ignored when MAC commands are present simultaneously in App-Payload and FOpts field

2.5.11. Incorrect MAC Commands

These tests ensure that a **DUT** behaves normally after it receives incorrect MAC commands from the **TCL**.

The invalid commands are:

- LinkADR with value out of spec - Payload: 0x0380000000
- Incomplete MAC command - Payload: 0x03010000
- Post Unknown MAC command ID - Payload: 0x7F

- Valid MAC command followed by invalid MAC commands – Payload: 0x0603010000

Verify:

- The **DUT** continue normal operation after receiving the invalid commands

2.5.11.a. Test Procedure Message Sequence Chart

Step	Procedure	Message Sequence		Test Purpose
		End Device - TCL	Message	
1	DUT sends Unconfirmed frame	→	FPort = 224	
2	The TCL sends Unconfirmed frame	←	CMD LinkADRReq Payload = 0x0380000000 FPort = 0	
3	DUT sends Unconfirmed frame	→	LinkADRAAns != OK	DUT continues normal operation after LinkADR with 'out of spec' Payload
4	The TCL sends Unconfirmed frame	←	Payload = 0x03010000 FPort = 0	
5	DUT sends Unconfirmed frame	→	No response	DUT continues normal operation after incomplete MAC command Payload
6	The TCL sends Unconfirmed frame	←	Payload = 0x7F FPort = 0	
7	DUT sends Unconfirmed frame	→	No response	DUT continues normal operation after unknown MAC command Payload
8	The TCL sends Unconfirmed frame	←	CMD1 DevStatusReq CMD2 Incomplete LinkADRReq (Payload = 0x0603010000) FPort = 0	
9	DUT sends Unconfirmed frame	→	CMD1 DevStatusAns	DUT continues normal operation after incomplete MAC command Payload
10	The TCL sends Unconfirmed frame	←	CMD1 LinkADRReq CMD2 <i>Invalid MAC Command</i> CMD3 NewChannelReq FPort = 0	

11	DUT sends Unconfirmed frame	→	CMD1 LinkADRAAns	DUT answers only the first MAC command and continues normal operation if the second MAC command is invalid
12	Wait for a maximum of $5 * 3$ (number of channels configured) = 15 uplink packets to be sent, i.e. until all channels are used at least once.	→	FPort = 224	Only default channels must be used at least once. The additional channel must not be added.

3. Notice of Use and Disclosure

Copyright © LoRa Alliance, Inc. (2020). All Rights Reserved.

The information within this document is the property of the LoRa Alliance (The Alliance) and its use and disclosure are subject to LoRa Alliance Corporate Bylaws, Intellectual Property Rights (IPR) Policy and Membership Agreements.

Elements of LoRa Alliance specifications may be subject to third party intellectual property rights, including without limitation, patent, copyright, or trademark rights (such a third party may or may not be a member of LoRa Alliance). The Alliance is not responsible and shall not be held responsible in any manner for identifying or failing to identify any or all such third party intellectual property rights.

This document and the information contained herein are provided on an AS IS basis and THE ALLIANCE DISCLAIMS ALL WARRANTIES EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO (A) ANY WARRANTY THAT THE USE OF THE INFORMATION HEREIN WILL NOT INFRINGE ANY RIGHTS OF THIRD PARTIES (INCLUDING WITHOUT LIMITATION ANY INTELLECTUAL PROPERTY RIGHTS INCLUDING PATENT, COPYRIGHT OR TRADEMARK RIGHTS) OR (B) ANY IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE OR NON-INFRINGEMENT.

IN NO EVENT WILL THE ALLIANCE BE LIABLE FOR ANY LOSS OF PROFITS, LOSS OF BUSINESS, LOSS OF USE OF DATA, INTERRUPTION OF BUSINESS, OR FOR ANY OTHER DIRECT, INDIRECT, SPECIAL OR EXEMPLARY, INCIDENTAL, PUNITIVE OR CONSEQUENTIAL DAMAGES OF ANY KIND, IN CONTRACT OR IN TORT, IN CONNECTION WITH THIS DOCUMENT OR THE INFORMATION CONTAINED HEREIN, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH LOSS OR DAMAGE.

The above notice and this paragraph must be included on all copies of this document that are made.

LoRa Alliance, Inc.
5177 Brandin Court,
Fremont, CA 94538 USA

Note: All Company, brand and product names may be trademarks that are the sole property of their respective owners.